Exam Copy-Paste Library — worked examples and deep dives

This file is a library of ready-to-paste fragments for the SB5-MAI reflective-report exam: complete smell and refactoring catalogs with minimal Java examples, a full CI/CD pipeline deep dive, justification sentences in the decision → rejected alternative → concrete benefit pattern, testing snippets (JUnit, AssertJ, Mockito, JGiven), Clean Code and Clean Architecture statements applied to lab work, and fill-in reflection templates for every course lab. Citations follow the course guides: (Refactoring1 p.X), (BetterCode p.X), (HighLevelRefactoring p.X), (CleanCode p.X), (DesignPrinciplesAndPatterns p.X), (ContinuousIntegration p.X), (BDD p.X), (CILab p.1). First-person sentences contain [placeholders] you replace with the names from your own JHotDraw repository. Content that goes past the slide decks is labelled "(beyond slides — practical knowledge)".

Code smell catalog — identify, example, fix, reflect

What is a code smell — the paste-ready definition

A code smell is a surface symptom in the source code — a recognisable pattern that usually, though not always, signals a deeper design problem and points to a candidate refactoring (Refactoring1 p.6–8). Smells are not bugs: the program may run perfectly while still smelling. Their value is as a trigger language — naming a smell tells you both what to look for and, usually, which refactoring to reach for. Fowler's catalog names 22 smells (Refactoring1 p.6–8); Kerievsky adds five more at the design level (HighLevelRefactoring p.12–13). The SIG guidelines turn several of the same smells into measurable thresholds — for example Long Method becomes "limit units to 15 lines of code" (BetterCode p.7) — so the same problem can be named as a smell, measured as a metric violation, or resolved with a pattern.

Paste-ready exam sentence: "A code smell is a surface indication in the code that usually corresponds to a deeper problem in the design; it is not a bug, but a heuristic that tells the maintainer where refactoring is likely to pay off (Refactoring1 p.6)."

How do you identify a code smell — the method

This is a named example exam question. A paste-ready answer:

I identify code smells in four complementary ways. First, by reading the code against the catalog: I know Fowler's named symptoms (Refactoring1 p.6–8) — a method I cannot see on one screen suggests Long Method, the same six lines in two places suggest Duplicated Code, a chain like `a.getB().getC().getD()` suggests Message Chains. Second, by measuring: the SIG guidelines give objective thresholds — units longer than 15 lines of code violate G1, more than 4 branch points violates G2, duplicated blocks of 6 or more lines violate G3, more than 4 parameters violates G4 (BetterCode p.10, p.16, p.23, p.25) — so a metric tool or a quick count flags candidates mechanically. Third, by listening to friction while changing the code: if one change forces edits in many classes (Shotgun Surgery) or one class keeps changing for unrelated reasons (Divergent Change), the change process itself exposes the smell. Fourth, by noticing compensations: a comment explaining a tangled block, a long explanatory variable name, or a test that is hard to write are all signs the underlying code smells (Refactoring1 p.8; BetterCode p.54). Identification is therefore both qualitative (pattern recognition against the catalog) and quantitative (threshold violations), and the smell name immediately suggests the refactoring that fixes it.

Duplicated Code

What it is. The same code structure appears in more than one place (Refactoring1 p.6). Fowler calls it the number-one smell. **How to spot it.** Identical or near-identical fragments in two methods of the same class, in sibling classes, or in unrelated classes; the SIG measure is a duplicated block of 6 or more lines of code appearing more than once (BetterCode p.23). Watch for copy-paste-modify history: same shape, different literals.

```
double basePrice = quantity * itemPrice;
if (basePrice > 1000) total = basePrice * 0.95;
else total = basePrice * 0.98;
// ...and the identical block again in another method
```

Fix. Extract Method to pull the shared fragment into one named method; Extract Superclass or Form Template Method when the duplication is across sibling classes; Pull Up Method into a common parent (Refactoring1 p.6). Why it matters: every fix must otherwise be made in several places, and one copy will inevitably be missed, producing bugs. **Reflection snippet.** "In my lab I found duplicated drawing logic in [ClassA] and [ClassB] in JHotDraw; I applied Extract Method and then Pull Up Method so the shared behaviour lives once in [Superclass], which means a future fix is made in exactly one place."

Long Method

What it is. A method that is too long and does many things at different levels of abstraction (Refactoring1 p.6). **How to spot it.** You scroll to read it; it needs comments to separate its phases; it mixes abstraction levels (business rules next to string formatting). Measurable form: SIG G1 says a unit over 15 lines of code is too long (BetterCode p.7, p.10) — count every non-empty, non-comment-only line.

```
public void printInvoice(Order order) {
    // 60 lines: header formatting, line-item loop,
    // discount calculation, tax rules, footer, logging...
}
```

Fix. Extract Method — each coherent fragment becomes a named method (the workhorse, Refactoring1 p.11); Replace Method with Method Object when tangled local variables block extraction (Refactoring1 p.13; BetterCode p.11). Short units are easier to test, analyze, and reuse (BetterCode p.8). **Reflection snippet.** "In my lab the method [methodName] in [Class] was [N] lines long, violating SIG guideline G1; I applied Extract Method repeatedly until each unit was under 15 lines, and each extracted name now documents one step of the algorithm."

Large Class

What it is. A class doing too much — too many fields and methods, a "god class" (Refactoring1 p.6). **How to spot it.** Many instance variables, many unrelated method groups, the class name is vague (`Manager` , `Util`), and different features import it for different reasons. It changes for many unrelated reasons, violating single responsibility.

```
class Editor { // 40 fields, 90 methods:
    // drawing, file I/O, undo history, printing,
    // preferences, clipboard, network export...
}
```

Fix. Extract Class to split out one coherent responsibility; Extract Subclass when some features are used only by some instances; Extract Interface to let clients depend on a slice (Refactoring1 p.6, p.17). Kerievsky routes the same smell to Command, State, or Interpreter at pattern level (HighLevelRefactoring p.12). **Reflection snippet.** "In my lab [Class] had accumulated both [responsibility 1] and [responsibility 2]; I applied Extract Class to move [responsibility 2] into [NewClass], so each class now changes for exactly one reason (SRP)."

Long Parameter List

What it is. A method takes too many parameters (Refactoring1 p.6). **How to spot it.** Call sites are unreadable; arguments get passed in the wrong order without compiler complaint; the same group of parameters travels together through several signatures. Measurable form: SIG G4 — more than 4 parameters per unit violates the guideline (BetterCode p.25).

```
void drawLabel(String text, int x, int y, int width,
               int height, Color fg, Color bg, Font font) { ... }
```

Fix. Introduce Parameter Object to bundle the cluster into one concept; Replace Parameter with Method when the receiver can compute the value itself; Preserve Whole Object (Refactoring1 p.6, p.40–41; BetterCode p.25, p.28). **Reflection snippet.** "In my lab [method] took [N] parameters; I introduced a [ParameterObject] class to carry them, which shortened every call site and gave the new concept a home for related behaviour such as [validation/derived value]."

Divergent Change

What it is. "When one class is commonly changed in different ways for different reasons" — one class, many unrelated reasons to change (Refactoring1 p.6). **How to spot it.** Look at the change history: if you say "when we add a new format I touch these three methods, but when we add a new database I touch those four" about the same class, it diverges. It is a single-responsibility violation visible across commits.

```
class ReportService {
    void renderHtml() { ... }   // changes when layout changes
    void renderPdf() { ... }
    void loadFromDb() { ... }   // changes when schema changes
    void saveToDb() { ... }
}
```

Fix. Extract Class so each resulting class changes for exactly one reason (Refactoring1 p.6). **Reflection snippet.** "In my lab [Class] changed both when [reason A] and when [reason B]; I split it with Extract Class so each change request now lands in exactly one class, shrinking every future impact set."

Shotgun Surgery

What it is. "When every time you make a kind of change, you have to make a lot of little changes to a lot of different classes" — the inverse of Divergent Change (Refactoring1 p.6). **How to spot it.** During impact analysis one logical change produces a long estimated impact set of small edits scattered across the codebase; you grep for a concept and find fragments of it everywhere. Changes are error-prone because one scattered edit is easy to forget.

```
// adding a new figure type requires edits in:
// FigureFactory, FigureRenderer, FigureSerializer,
// ToolPalette, FigureValidator ... every release
```

Fix. Move Method and Move Field to gather the scattered responsibility into one class; sometimes Inline Class to merge fragments (Refactoring1 p.6). **Reflection snippet.** "In my lab adding [kind of change] forced small edits in [N] classes; I used Move Method to centralise the behaviour in [Class], so the next change of that kind was a one-class edit — my impact analysis before and after shows the estimated impact set shrinking from [N] to [1–2] classes."

Feature Envy

What it is. "A method that seems more interested in a class other than the one it actually is in" — it calls many methods or fields of another object (Refactoring1 p.6). **How to spot it.** Count the references inside the method: if it touches another object's getters several times and its own state rarely or never, it envies that class. The behaviour lives apart from the data it uses, increasing coupling.

```
class OrderPrinter {
    String describe(Order o) {
        return o.getId() + " " + o.getCustomer().getName()
             + " " + o.getTotal() + " " + o.getDate();
    }
}
```

Fix. Move Method to the class whose data it uses most; if only part of the method envies, Extract Method first, then Move Method (Refactoring1 p.6, p.15). **Reflection snippet.** "In my lab [method] in [ClassA] read four fields of [ClassB] and none of its own; I moved it into [ClassB] with Move Method, putting behaviour next to the data it uses and cutting the coupling between the two classes."

Data Clumps

What it is. "Bunches of data that regularly appear together" — the same group of fields or parameters recurring in several places (Refactoring1 p.6). **How to spot it.** The same three or four names travel together through signatures and field lists (`x, y, width, height`; `street, city, zip`). Delete-one test: if removing one of the values makes the rest meaningless, they are one concept missing its class.

```
void move(int x, int y) { /* ... */ }
void resize(int x, int y, int w, int h) { /* ... */ }
void paint(Graphics g, int x, int y, int w, int h) { /* ... */ }
```

Fix. Extract Class for clumped fields; Introduce Parameter Object for clumped parameters (Refactoring1 p.6) — this is also the named fix for SIG G4 (BetterCode p.25). **Reflection snippet.** "In my lab the values [a, b, c] travelled together through [N] signatures; I extracted a [Concept] class for them, which shortened the signatures and immediately attracted related behaviour such as [method] into the new class."

Primitive Obsession

What it is. "Excessive use of primitives, due to reluctance to use small objects for small tasks" — a String for a phone number, an int for money (Refactoring1 p.7). **How to spot it.** Validation and formatting of the same value scattered across the codebase; type codes as ints or Strings; comments explaining what a primitive means; groups of primitives simulating a structure.

```
String phone;           // validated in 4 places
int customerType;       // 1=NORMAL, 2=VIP, 3=STAFF
double priceInDkk;      // currency by convention only
```

Fix. Replace Data Value with Object; Replace Type Code with Class or with Subclasses (Refactoring1 p.7, p.25). Kerievsky routes it to Replace Type Code with Class, Replace Implicit Tree with Composite, or Encapsulate Composite with Builder (HighLevelRefactoring p.12–13). **Reflection snippet.** "In my lab [value] was a bare [String/int] validated in [N] places; I introduced a [ValueClass] that validates once in its constructor, so an invalid [value] can no longer exist anywhere in the system."

Switch Statements

What it is. Type-switching logic — switch or if-else chains on a type code that should be polymorphism, especially when the same switch recurs in several places (Refactoring1 p.7). **How to spot it.** A switch on an object's type or a type-code field; the same case structure duplicated in multiple methods; adding a new kind means hunting every switch — an Open/Closed Principle violation (DesignPrinciplesAndPatterns p.5).

```
double area(Shape s) {
    switch (s.getType()) {
        case CIRCLE:    return s.r() * s.r() * Math.PI;
        case RECTANGLE: return s.w() * s.h();
        default: throw new IllegalArgumentException();
    }
}
```

Fix. Replace Conditional with Polymorphism — each subclass owns its case and the original method becomes abstract (Refactoring1 p.35); also BetterCode G2's recommended refactoring (BetterCode p.18). Kerievsky: Replace Conditional Dispatcher with Command, or Move Accumulation to Visitor (HighLevelRefactoring p.12–13). **Reflection snippet.** "In my lab the type-switch on [typeCode] recurred in [N] methods; I replaced it with polymorphism so each [Subclass] computes its own [behaviour] — adding a new type now means adding a class, not editing every switch."

Parallel Inheritance Hierarchies

What it is. "Where every time you make a subclass of one class, you also have to make a subclass of another" (Refactoring1 p.7) — a special case of Shotgun Surgery. **How to spot it.** Two hierarchies with rhyming prefixes (`CircleFigure` / `CircleTool`, `SquareFigure` / `SquareTool`); creating a class in one always forces a twin in the other; the hierarchies must be kept in lock-step.

```
abstract class Figure { }   class CircleFigure extends Figure {}
abstract class Tool   { }   class CircleTool   extends Tool   {}
// every new Figure subclass demands a new Tool subclass
```

Fix. Move Method and Move Field to make one hierarchy refer to the other generically; ultimately Tease Apart Inheritance for the deep case (Refactoring1 p.7, p.52). **Reflection snippet.** "In my lab every new [A] subclass forced a matching [B] subclass; I moved the [B]-side behaviour into [A] (Move Method), so one hierarchy disappeared and a new variant is now a single new class."

Lazy Class

What it is. "A class that isn't doing enough to justify its maintenance" (Refactoring1 p.7). **How to spot it.** A class with one trivial method or only forwarding logic; a subclass that barely differs from its parent; classes left behind after refactoring drained them. Every class costs reading, navigation, and maintenance — one that earns too little is pure overhead.

```
class NameHolder {           // its only job
    private String name;
    String getName() { return name; }
}
```

Fix. Inline Class — fold it into its user; Collapse Hierarchy if it is a thin subclass (Refactoring1 p.7, p.46). Kerievsky adds Inline Singleton (HighLevelRefactoring p.12). **Reflection snippet.** "After my earlier refactoring, [Class] no longer pulled its weight; I applied Inline Class to fold it into [User], deleting an indirection that cost readers a file-jump and bought nothing."

Speculative Generality

What it is. "Classes and features have been added just because a need for them may arise someday" — abstraction with no current user, a YAGNI violation (Refactoring1 p.7). **How to spot it.** Abstract classes with a single implementation; unused parameters and hooks; methods named for generality with one caller; test code is the only user. Unused flexibility is dead weight that confuses readers.

```
abstract class AbstractExporterFactoryBase {
    // one concrete subclass exists; the "flexibility"
    // has never been exercised
}
```

Fix. Collapse Hierarchy, Inline Class, Remove Parameter, Rename Method to bring the code back to what is actually needed (Refactoring1 p.7). **Reflection snippet.** "In my lab [abstraction] existed for a future that never came — it had exactly one implementation; I collapsed it (Collapse Hierarchy) because unused generality is a reading cost with no benefit, and version control means I can resurrect it if the need ever becomes real."

Temporary Field

What it is. "An instance variable that is set only in certain circumstances" — otherwise empty or null (Refactoring1 p.7). **How to spot it.** Fields that are only meaningful during one algorithm; null checks guarding field use; the constructor leaves fields unset and some method pair sets and clears them. Readers cannot tell when the field is valid — it is a hidden conditional.

```
class PathFinder {
    private List<Node> scratchQueue;  // only non-null
    private Node scratchBest;         // while solve() runs
}
```

Fix. Extract Class to give the temporary fields and their algorithm a home (often a method object); Introduce Null Object for the "sometimes absent" case (Refactoring1 p.7). **Reflection snippet.** "In my lab [field] in [Class] was only valid while [method] ran; I extracted a [Helper] class whose lifetime equals the computation, so every field in [Class] is now always valid."

Message Chains

What it is. "Transitive visibility chains" — a client navigates `a.getB().getC().getD()` (Refactoring1 p.7). **How to spot it.** Trains of getters in one expression; the client is coupled to the entire navigation structure, so any intermediate change breaks it. This is also a Law of Demeter violation — talking to strangers (CleanCode p.61–64).

```
String mgr = order.getCustomer().getDepartment()
                  .getManager().getName();
```

Fix. Hide Delegate — create a method on the server that hides the hop (`order.getManagerName()`); or Extract Method on the chain's use and Move Method it down the chain (Refactoring1 p.7, p.18). **Reflection snippet.** "In my lab [Client] reached through [N] objects to get [value]; I applied Hide Delegate so [Server] exposes [method] directly, and [Client] no longer breaks when the intermediate structure changes."

Middle Man

What it is. "Excessive delegation" — a class where most methods just forward to another class (Refactoring1 p.8). **How to spot it.** Open the class: if more than half its methods are one-line delegations to the same field, the wrapper adds indirection without value. Note the deliberate tension with Message Chains — Hide Delegate creates middle men; you balance the two smells.

```
class Department {
    private Manager manager;
    String managerName() { return manager.getName(); }
    int managerPhone()  { return manager.getPhone(); }
    // ...ten more pure forwards
}
```

Fix. Remove Middle Man — let clients talk to the delegate directly (Refactoring1 p.8, p.19). **Reflection snippet.** "In my lab [Class] forwarded [N] of its [M] methods straight to [Delegate]; I removed the middle man so clients call [Delegate] directly — the indirection hid nothing and cost a class."

Inappropriate Intimacy

What it is. "Excessive interaction and coupling" between two classes — they poke into each other's private parts (Refactoring1 p.8). **How to spot it.** Bidirectional references; one class reading and writing another's fields or relying on its internals; subclasses exploiting superclass internals. Tight two-way coupling makes either class impossible to change alone.

```
class A { B b; void doIt() { b.internalList.add(this); } }
class B { List<A> internalList; A lastA; }
```

Fix. Move Method/Move Field to put behaviour where the data is; Extract Class for the shared part; Change Bidirectional Association to Unidirectional; Replace Inheritance with Delegation when the intimacy comes through extends (Refactoring1 p.8). **Reflection snippet.** "In my lab [ClassA] and [ClassB] manipulated each other's internals; I extracted the shared [concern] into [NewClass] and made the remaining association one-directional, so each class can now change without breaking the other."

Alternative Classes with Different Interfaces

What it is. "Classes that do the same thing but have different interfaces for what they do" (Refactoring1 p.8). **How to spot it.** Two classes solve the same problem but with different method names and signatures; clients cannot treat them uniformly or swap one for the other.

```
class PngWriter  { void write(File f)      { /* ... */ } }
class JpegSaver  { void store(String path) { /* ... */ } }
```

Fix. Rename Method and Move Method to align the interfaces until they match, then possibly Extract Interface or Extract Superclass; Kerievsky's pattern route is Unify Interfaces with Adapter (Refactoring1 p.8; HighLevelRefactoring p.12). **Reflection snippet.** "In my lab [ClassA] and [ClassB] did the same job behind different method names; I renamed and aligned their interfaces and extracted [Interface], so clients now program to the role and either implementation can be swapped in."

Incomplete Library Class

What it is. A library class is missing methods you need, but you cannot edit its source (Refactoring1 p.8). **How to spot it.** Repeated awkward workarounds around the same third-party type; static helper methods taking the library object as first argument scattered across the codebase.

```
// java.util.Date lacks nextDay(); helpers appear everywhere:
static Date nextDay(Date d) { /* ... */ }  // in 3 different classes
```

Fix. Introduce Foreign Method when you need one method (client-side method taking the server instance as first argument); Introduce Local Extension when several methods are needed (subclass or wrapper bundling them) (Refactoring1 p.8, p.21–22). **Reflection snippet.** "In my lab the library class [LibClass] lacked [operation]; since I cannot modify a third-party class, I introduced a local extension [MyLibClass] wrapping it, so all our additions live in one announced place instead of being scattered as ad-hoc helpers."

Data Class

What it is. "Classes that have fields, getting and setting methods for the fields, and nothing else" — anaemic data carriers with no behaviour (Refactoring1 p.8). **How to spot it.** All getters and setters, no domain methods; the logic that uses the data lives elsewhere (often producing Feature Envy in other classes); public mutable collections handed out raw.

```
class Invoice {
    private List<Line> lines;
    public List<Line> getLines() { return lines; }
    public void setLines(List<Line> l) { lines = l; }
}
```

Fix. Move Method to pull the behaviour that belongs with the data into the class; Encapsulate Field and Encapsulate Collection to stop external mutation; Hide Methods that no longer need to be public (Refactoring1 p.8). **Reflection snippet.** "In my lab [DataClass] was pure getters and setters while [OtherClass] computed [logic] from its fields; I moved [logic] into [DataClass] (Move Method) and returned a read-only view of its collection (Encapsulate Collection), turning a passive record into a real object."

Refused Bequest

What it is. "When subclasses do not fulfill the commitments of their superclasses" — inheriting but ignoring or overriding most of the parent (Refactoring1 p.8). **How to spot it.** Overrides that throw `UnsupportedOperationException`; subclasses using only a sliver of the inherited interface; "is-a" reads false aloud. This breaks Liskov substitution — clients holding the supertype get surprises (DesignPrinciplesAndPatterns p.8).

```
class Stack extends ArrayList<Object> {
    // inherits add(int, e), remove(int)... all of which
    // break the stack discipline; we "refuse" them
}
```

Fix. Replace Inheritance with Delegation — hold the former parent as a field and forward only what is truly needed; or Push Down Method/Field so the parent only promises what all children honour (Refactoring1 p.8, p.49). **Reflection snippet.** "In my lab [Subclass] extended [Superclass] but refused most of its contract; I replaced the inheritance with delegation, so [Subclass] now exposes only the operations it genuinely supports and Liskov substitution holds again."

Comments (as a smell)

What it is. "When comments are used to compensate for bad code" — a comment explaining a tangled block (Refactoring1 p.8). The smell is not commenting itself; it is the comment as deodorant. **How to spot it.** Comments that narrate *what* the next block does (rather than *why*); commented-out code; comments that have drifted from the code they describe. CleanCode's taxonomy of bad comments: redundant, misleading, mandated, journal, noise, position markers, commented-out code (CleanCode p.38–52).

```
// check if employee is eligible for full benefits
if ((emp.flags & HOURLY_FLAG) != 0 && emp.age > 65) { /* ... */ }
```

Fix. Extract Method with a self-documenting name (`if (emp.isEligibleForFullBenefits())`) so the code explains itself and the comment becomes unnecessary (Refactoring1 p.8; CleanCode p.35). The fix is not "delete all comments" — good comments (legal, informative, intent-explaining, warnings, javadoc on public APIs) survive (CleanCode p.35–37). **Reflection snippet.** "In my lab a comment in [Class] explained a [N]-line conditional; I extracted the block into [wellNamedMethod] so the name now says what the comment said, and the comment — which could have drifted out of date — is gone."

Combinatorial Explosion (Kerievsky)

What it is. One of Kerievsky's five additional smells: many code variants multiplying out to express what is really a small implicit language of rules or queries (HighLevelRefactoring p.12–13). Each new combination of options spawns yet another method or class. **How to spot it.** Method families whose names enumerate combinations; parallel variants growing multiplicatively as options are added.

```
List<User> findActiveByName(String n) { /* ... */ }
List<User> findActiveByNameAndCity(String n, String c) { /* ... */ }
List<User> findInactiveByCity(String c) { /* ... */ } // and on and on
```

Fix. Replace Implicit Language with Interpreter (Ker05 p.269) — make the rule language explicit as composable query/specification objects, so combinations are composed at runtime instead of enumerated in code (HighLevelRefactoring p.12). **Reflection snippet.** "In my lab the [finder] methods multiplied with every new filter; I made the implicit query language explicit as composable [Specification] objects, so a new combination is composed, not coded."

Conditional Complexity (Kerievsky)

What it is. Kerievsky's broader umbrella over complicated conditional logic — conditionals that grow, nest, and accumulate special cases (HighLevelRefactoring p.12–13). It generalises Fowler's Switch Statements to all tangled branching. **How to spot it.** Nested if-pyramids; boolean flags steering behaviour; SIG G2 violation — more than 4 branch points per unit, i.e. cyclomatic complexity above 5 (BetterCode p.13, p.16).

```
if (user != null) {
    if (user.isActive()) {
        if (order.getTotal() > limit && !user.isVip()) { /* ... */ }
        else { /* ... */ }
```

```
    } else { /* ... */ }
}
```

Fix. All four of Kerievsky's fixes are behavioural patterns: Replace Conditional Logic with Strategy, Move Embellishment to Decorator, Replace State-Altering Conditionals with State, Introduce Null Object (HighLevelRefactoring p.12). At the Fowler level: Decompose Conditional and Replace Nested Conditional with Guard Clauses (Refactoring1 p.32, p.34). **Reflection snippet.** "In my lab [method] had cyclomatic complexity [N], violating G2; I introduced guard clauses for the special cases and moved the varying [policy] behind a Strategy, bringing every unit back under 4 branch points."

Indecent Exposure (Kerievsky)

What it is. Classes or constructors visible to clients that should not see them — implementation types leaking out of their package (HighLevelRefactoring p.12–13). **How to spot it.** Public concrete classes that only exist to serve one abstraction; client code calling `new` on implementation classes; the package's public surface is its whole contents.

```
// clients write:
Descriptor d = new AttributeDescriptor(/* ... */); // impl class public
// instead of asking a factory for a Descriptor
```

Fix. Encapsulate Classes with Factory (Ker05 p.80) — make the implementation classes package-private and hand clients a factory that returns them typed as the public abstraction (HighLevelRefactoring p.12). **Reflection snippet.** "In my lab clients instantiated [ImplClass] directly; I encapsulated the implementation classes behind [Factory], shrinking the package's public surface so I can now change implementations without touching any client."

Oddball Solution (Kerievsky)

What it is. The same problem solved one way almost everywhere but differently in one spot — the inconsistent outlier (HighLevelRefactoring p.12–13). **How to spot it.** Reading for convention: nine call sites use the standard helper, the tenth hand-rolls its own; two libraries doing the same job coexist because one corner of the code never migrated.

```
// everywhere: money handled via Money class
// in ReportExporter only: double amount + manual rounding
```

Fix. Unify Interfaces with Adapter (Ker05 p.247) — adapt the odd solution to the common interface, then retire it, so there is one way to do each thing (HighLevelRefactoring p.12). **Reflection snippet.** "In my lab [Class] solved [problem] differently from the rest of the codebase; I adapted it to the common [interface] and removed the oddball, because two solutions to one problem doubles the learning and maintenance cost."

Solution Sprawl (Kerievsky)

What it is. The code implementing one responsibility has sprawled across multiple classes — classically, object-creation knowledge smeared over the codebase (HighLevelRefactoring p.12–13). The sibling of Shotgun Surgery, observed while reading rather than while changing. **How to spot it.** To understand how one thing works (say, how a Figure gets created and wired), you must read five classes; creation logic, configuration, and registration each live somewhere else.

```
// FigureFactory makes it, ToolPalette configures it,
// Editor registers listeners, AppInit wires undo support —
// "creating a figure" is implemented in four places
```

Fix. Move Creation Knowledge to Factory (Ker05 p.68) — gather the sprawled creation responsibility into one Factory (HighLevelRefactoring p.12). **Reflection snippet.** "In my lab creating a [Thing] involved code in [N] classes; I moved all creation knowledge into [Factory], so the answer to 'how is a [Thing] made?' is now one class long."

Smell → refactoring quick lookup table

All 27 course smells with their primary fixes, for fast mid-exam lookup (Refactoring1 p.6–8; HighLevelRefactoring p.12–13):

Smell	Primary fix	Also
Duplicated Code	Extract Method	Extract Superclass, Form Template Method, Pull Up Method
Long Method	Extract Method	Replace Method with Method Object; SIG G1
Large Class	Extract Class	Extract Subclass, Extract Interface
Long Parameter List	Introduce Parameter Object	Replace Parameter with Method; SIG G4
Divergent Change	Extract Class	—
Shotgun Surgery	Move Method / Move Field	Inline Class
Feature Envy	Move Method	Extract Method then Move Method
Data Clumps	Extract Class / Introduce Parameter Object	—
Primitive Obsession	Replace Data Value with Object	Replace Type Code with Class/Subclasses
Switch Statements	Replace Conditional with Polymorphism	Command, Visitor (Kerievsky); SIG G2
Parallel Inheritance Hierarchies	Move Method / Move Field	Tease Apart Inheritance
Lazy Class	Inline Class	Collapse Hierarchy; Inline Singleton
Speculative Generality	Collapse Hierarchy, Inline Class	Remove Parameter, Rename Method
Temporary Field	Extract Class	Introduce Null Object
Message Chains	Hide Delegate	Extract Method + Move Method
Middle Man	Remove Middle Man	—
Inappropriate Intimacy	Move Method/Field, Extract Class	Change Bidirectional Association to Unidirectional
Alternative Classes with Different Interfaces	Rename Method / Move Method	Unify Interfaces with Adapter
Incomplete Library Class	Introduce Foreign Method	Introduce Local Extension
Data Class	Move Method	Encapsulate Field/Collection
Refused Bequest	Replace Inheritance with Delegation	Push Down Method/Field
Comments	Extract Method (self-documenting name)	Rename Method

Smell	Primary fix	Also
Combinatorial Explosion (Ker)	Replace Implicit Language with Interpreter	—
Conditional Complexity (Ker)	Strategy / Decorator / State / Null Object	Decompose Conditional, Guard Clauses
Indecent Exposure (Ker)	Encapsulate Classes with Factory	—
Oddball Solution (Ker)	Unify Interfaces with Adapter	—
Solution Sprawl (Ker)	Move Creation Knowledge to Factory	—

Refactoring pattern catalog — before/after

What is meant by a refactoring pattern — the paste-ready answer

This is a named example exam question. A paste-ready answer:

Refactoring is "a change made to the internal structure of software to make it easier to understand and cheaper to modify without changing its observable behavior" (Refactoring1, after Fowler). A refactoring pattern is a named, reusable recipe for such a change: it has a name (for example Extract Method), a context describing the problem it solves (a code smell such as Long Method), a transformation with before-and-after structure, and mechanics — the safe small steps that carry the code from one to the other while tests keep passing. Fowler's catalog groups roughly fifty such patterns into seven categories by the kind of structural problem each solves (Refactoring1 p.9). The word "pattern" carries the same meaning as in design patterns — Alexander's three-part rule of context, problem, and solution (HighLevelRefactoring p.7) — and Kerievsky's "Refactoring to Patterns" closes the circle by giving composite refactorings whose targets are design patterns themselves (HighLevelRefactoring p.12–13). The value of naming refactorings is the same as naming smells: a shared vocabulary lets me say "I applied Replace Conditional with Polymorphism to the type switch in [Class]" and any reader knows exactly what transformation took place, why, and what the code now looks like.

The seven refactoring categories — the map of the catalog

Fowler groups the catalog into seven families by the kind of structural problem each solves (Refactoring1 p.9). Knowing the family lets you find the right move once the mess is named:

1. **Composing Methods** — fixing how code is packaged into methods (Extract Method, Inline Method, Replace Method with Method Object). For method-level structure: short, well-named, single-purpose units.
2. **Moving Features Between Objects** — reassigning responsibilities (Move Method/Field, Extract/Inline Class, Hide Delegate, Remove Middle Man, Introduce Foreign Method/Local Extension). For Feature Envy, Large Class, Inappropriate Intimacy.
3. **Organizing Data** — making data easier to work with (Encapsulate Field/Collection, Replace Data Value with Object, Replace Array with Object, Self Encapsulate Field, Duplicate Observed Data, Replace Subclass with Fields). For Primitive Obsession, Data Class, exposed state.
4. **Simplifying Conditional Expressions** — making conditional logic less error-prone (Decompose Conditional, Consolidate Conditional Expression, Guard Clauses, Replace Conditional with Polymorphism, Introduce Null Object). For complex branching, the G2 target.
5. **Making Method Calls Simpler** — making interfaces easy to understand and use (Separate Query from Modifier, Parameterize Method, Replace Parameter with Method, Introduce Parameter Object). For Long

Parameter List and unsafe APIs.

6. **Dealing with Generalization** — moving features around an inheritance hierarchy (Pull Up Constructor Body, Extract Super/Subclass/Interface, Collapse Hierarchy, Form Template Method, Replace Inheritance with Delegation and its inverse). For sibling duplication and Refused Bequest.
7. **Big Refactorings** — large-scale, long-running restructurings (Tease Apart Inheritance, Convert Procedural Design to Objects, Separate Domain from Presentation, Extract Hierarchy).

Extract Method (Fowler 110)

Definition. "You have a code fragment that can be grouped together. Turn the fragment into a method whose name explains the purpose of the method" (Refactoring1 p.11). The single most common refactoring.

Before:

```
void printOwing() {
    printBanner();
    // print details
    System.out.println("name: " + name);
    System.out.println("amount: " + getOutstanding());
}
```

After:

```
void printOwing() {
    printBanner();
    printDetails(getOutstanding());
}
void printDetails(double outstanding) {
    System.out.println("name: " + name);
    System.out.println("amount: " + outstanding);
}
```

When/why. Whenever a block needs a comment, a method exceeds 15 lines (G1, BetterCode p.10), or the same fragment appears twice. The new name documents intent. **Mechanics:** create a new method named for *what* the fragment does; copy the fragment; pass needed locals as parameters and return the computed value; replace the fragment with a call; compile and test after each step [Fowler99]. **Reflection snippet.** "I extracted [fragment] from [longMethod] into [newMethod] instead of leaving an explanatory comment, because a named method cannot drift out of date the way a comment can, it shrank the unit below the 15-line G1 threshold, and it became reusable from [otherMethod]."

Inline Method (Fowler 117)

Definition. "A method's body is just as clear as its name. Put the method's body into the body of its callers and remove the method" (Refactoring1 p.11). The inverse of Extract Method.

Before:

```
int getRating() { return moreThanFiveLateDeliveries() ? 2 : 1; }
boolean moreThanFiveLateDeliveries() { return lateDeliveries > 5; }
```

After:

```
int getRating() { return lateDeliveries > 5 ? 2 : 1; }
```

When/why. When the indirection earns nothing — the body is as obvious as the call — or to undo over-eager extraction before regrouping a method differently. **Mechanics:** check the method is not polymorphic (no subclass overrides it); replace each call with the body; remove the method; compile and test [Fowler99]. **Reflection snippet.** "I inlined [tinyMethod] back into [caller] rather than keeping the extra hop, because the name added no information beyond the expression itself, and removing the needless indirection made the call site read in one glance."

Replace Method with Method Object (Fowler 135)

Definition. "You have a long method that uses local variables in such a way that you cannot apply Extract Method. Turn the method into an object so that all the local variables become fields on that object. It can then be decomposed into other methods on the same object" (Refactoring1 p.13). Listed as a G1 fix (BetterCode p.11).

Before:

```
double price() {
    double primaryBase, secondaryBase, tertiaryBase;
    // long computation entangling all three locals...
}
```

After:

```
double price() { return new PriceCalculator(this).compute(); }
class PriceCalculator {
    private double primaryBase, secondaryBase, tertiaryBase;
    double compute() { /* now decomposable into small methods */ }
}
```

When/why. The escape hatch when tangled locals block plain Extract Method — promoting locals to fields lets you finally split the monster. **Mechanics:** create a class named after the method; give it a field for the source object and one per local; add a constructor and a compute method containing the old body; delegate the old method to it; then Extract Method freely inside the new class [Fowler99]. **Reflection snippet.** "Because [method]'s locals were too entangled to extract directly, I turned it into a [MethodObject] class instead of forcing parameter-heavy extractions, which let me decompose the algorithm into [N] small named steps without threading five arguments through every one."

Move Method (Fowler 142)

Definition. "A method is, or will be, using or used by more features of another class than the class on which it is defined. Create a new method with a similar body in the class it uses most; either turn the old method into a simple delegation, or remove it altogether" (Refactoring1 p.15).

Before:

```
class Account {
    double overdraftCharge() {
        if (type.isPremium()) { /* uses type's data throughout */ }
        return 1.7 * daysOverdrawn;
    }
}
```

After:

```
class AccountType {
    double overdraftCharge(int daysOverdrawn) { /* lives with its data */ }
}
class Account {
    double overdraftCharge() { return type.overdraftCharge(daysOverdrawn); }
}
```

When/why. The cure for Feature Envy — put behaviour next to the data it uses; also helps Shotgun Surgery and Inappropriate Intimacy. **Mechanics:** examine what the method uses; declare it in the target class; copy and adapt the body; delegate from the old home; decide whether to keep or remove the delegation; compile and test [Fowler99]. **Reflection snippet.** "I moved [method] from [ClassA] to [ClassB] rather than passing [ClassB]'s data across as parameters, because the method referenced [ClassB]'s features [N] times and its own none — after the move the coupling dropped and the parameter list disappeared."

Move Field (Fowler 146)

Definition. "A field is, or will be, used by another class more than the class on which it is defined. Create a new field in the target class, and change all its users" (Refactoring1 p.15).

Before:

```
class Account { private double interestRate; }
class AccountType { /* methods keep asking account for the rate */ }
```

After:

```
class AccountType { private double interestRate; }
class Account { double rate() { return type.getInterestRate(); } }
```

When/why. The data-level counterpart of Move Method — usually part of re-homing responsibilities during Extract Class or while fixing Feature Envy. **Mechanics:** encapsulate the field if it is public; create the field (and accessors) in the target; redirect all users; remove the old field [Fowler99]. **Reflection snippet.** "I moved [field] into [TargetClass] together with the methods that used it, instead of leaving data and behaviour split across two classes, so the concept now lives in one place and the getter chain between the classes vanished."

Extract Class (Fowler 149)

Definition. "You have one class doing work that should be done by two. Create a new class and move the relevant fields and methods from the old class into the new class" (Refactoring1 p.17).

Before:

```
class Person {
    private String name;
    private String officeAreaCode, officeNumber;
    String getTelephoneNumber() { return "(" + officeAreaCode + ") " + officeNumber; }
}
```

After:

```
class Person {
    private String name;
```

```
    private TelephoneNumber officeTelephone = new TelephoneNumber();
}
class TelephoneNumber {
    private String areaCode, number;
    String toString() { return "(" + areaCode + ") " + number; }
}
```

When/why. The primary cure for Large Class, Data Clumps, Divergent Change, and Temporary Field — split so each class changes for one reason (SRP). **Mechanics:** decide how to split responsibilities; create the new class; link old to new; Move Field then Move Method one element at a time, testing after each; review and trim interfaces [Fowler99]. **Reflection snippet.** "I extracted [NewClass] out of [GodClass] instead of just regrouping methods inside it, because the two responsibilities changed at different times for different reasons; after the split, the change request [CR] touched only [NewClass] and its tests."

Inline Class (Fowler 154)

Definition. "A class isn't doing very much. Move all its features into another class and delete it" (Refactoring1 p.17). Inverse of Extract Class.

Before:

```
class TelephoneNumber { String areaCode, number; }
class Person { TelephoneNumber phone; }  // phone barely used
```

After:

```
class Person { String areaCode, number; }
```

When/why. Fixes Lazy Class — often after other refactorings drained the class. **Mechanics:** declare the absorbed class's public methods on the absorber, delegating; redirect all clients to the absorber; move fields and methods across; delete the husk [Fowler99]. **Reflection snippet.** "After moving [behaviour] elsewhere, [SmallClass] held one field and no logic, so I inlined it into [Owner] rather than maintaining a file, a constructor, and an import for a class that no longer earned its keep."

Hide Delegate (Fowler 157)

Definition. "A client is calling a delegate class of an object. Create methods on the server to hide the delegate" (Refactoring1 p.18).

Before:

```
manager = john.getDepartment().getManager();
```

After:

```
manager = john.getManager();
// inside Person: Person getManager() { return department.getManager(); }
```

When/why. Fixes Message Chains — the client no longer knows about Department, so a change to the intermediate structure cannot break it. **Mechanics:** for each delegate method the client uses, create a delegating method on the server; switch clients to it; if no client needs the delegate any more, hide the accessor [Fowler99]. Balance against Remove Middle Man — over-applying creates Middle Man. **Reflection**

snippet. "I hid [Delegate] behind a method on [Server] instead of letting [Client] navigate the chain, because the chain coupled [Client] to the whole object structure; the trade-off I accepted is one extra forwarding method, which is cheaper than a ripple through every client when the structure changes."

Remove Middle Man (Fowler 160)

Definition. "A class is doing too much simple delegation. Get the client to call the delegate directly" (Refactoring1 p.19). Inverse of Hide Delegate.

Before:

```
class Person {
    Manager getManager() { return department.getManager(); }
    // ...a dozen more pure forwards to department
}
```

After:

```
manager = john.getDepartment().getManager(); // client goes direct
```

When/why. When the server has become a pure forwarding shell — the indirection costs more than it hides. You pick between this and Hide Delegate based on which smell dominates: exposed chains vs delegation bloat. **Mechanics:** add an accessor for the delegate; for each forwarding method, switch clients to call the delegate directly, then delete the forward [Fowler99]. **Reflection snippet.** "I removed the forwarding layer in [Class] rather than keep adding a delegating method per new feature, because [N] of its methods were one-line forwards — clients now call [Delegate] directly and [Class] shrank to its real responsibility."

Introduce Foreign Method (Fowler 162)

Definition. "A server class you are using needs an additional method, but you can't modify the class. Create a method in the client class with an instance of the server class as its first argument" (Refactoring1 p.21).

Before:

```
Date newStart = new Date(previousEnd.getYear(),
        previousEnd.getMonth(), previousEnd.getDate() + 1);
```

After:

```
Date newStart = nextDay(previousEnd);
private static Date nextDay(Date arg) {   // foreign method; should be on Date
    return new Date(arg.getYear(), arg.getMonth(), arg.getDate() + 1);
}
```

When/why. Fixes Incomplete Library Class when exactly one method is missing. Mark it as a foreign method so it can migrate if the library ever gains the feature. **Mechanics:** create the method in the client taking the server instance as first argument; comment it as "foreign method, should be in server" [Fowler99]. **Reflection snippet.** "Since I cannot edit [LibraryClass], I wrote [helper] as a foreign method taking the instance as its first argument, instead of scattering the same three lines at every call site — one announced workaround beats five anonymous ones."

Introduce Local Extension (Fowler 164)

Definition. "A server class you are using needs several additional methods, but you can't modify the class. Create a new class that contains these extra methods. Make this extension class a subclass or a wrapper of the original" (Refactoring1 p.22).

Before:

```
// nextDay(d), previousDay(d), isWeekend(d) helpers
// spread across three client classes
```

After:

```
class MfDate extends Date {       // subclass form of local extension
    Date nextDay() { /* ... */ }
    boolean isWeekend() { /* ... */ }
}
```

When/why. Fixes Incomplete Library Class when several methods are needed — bundle them into a proper subclass or wrapper instead of sprinkling foreign methods. **Mechanics:** create the extension as subclass or wrapper; add converting constructors; add the new features; replace uses of the original where the extras are needed [Fowler99]. **Reflection snippet.** "When my date helpers reached [N], I consolidated them into a local extension [MyDate] rather than keeping foreign methods in three clients, because a single extension class gives the additions one discoverable home and a place for their tests."

Self Encapsulate Field (Fowler 171)

Definition. "You are accessing a field directly, but the coupling to the field is becoming awkward. Create getting and setting methods for the field and use only those to access the field" (Refactoring1 p.24).

Before:

```
private int low, high;
boolean includes(int arg) { return arg >= low && arg <= high; }
```

After:

```
private int low, high;
int getLow() { return low; }
int getHigh() { return high; }
boolean includes(int arg) { return arg >= getLow() && arg <= getHigh(); }
```

When/why. When a subclass needs to override how the value is computed, or further refactoring needs one access funnel — lazy initialisation and validation also get a single home. **Mechanics:** create accessors; replace every direct use, including in the declaring class; compile and test [Fowler99]. **Reflection snippet.** "I routed all access to [field] through accessors instead of touching the field directly, because the subclass [Sub] needed to redefine how the value is derived — with one access funnel the override was a one-method change."

Replace Data Value with Object (Fowler 175)

Definition. "You have a data item that needs additional data or behavior. Turn the data item into an object" (Refactoring1 p.25).

Before:

```
class Order { private String customer; }
```

After:

```
class Order { private Customer customer; }
class Customer {
    private final String name;
    Customer(String name) { this.name = name; }
    String getName() { return name; }
}
```

When/why. The direct cure for Primitive Obsession — the moment a "simple" value needs validation, formatting, or related data, give it a class; behaviour then accretes in the right place. **Mechanics:** create the value class with the primitive as constructor argument and a getter; change the owner's field type; adjust accessors; compile and test [Fowler99]. **Reflection snippet.** "I promoted [primitive value] to a [ValueClass] instead of validating the raw [String/int] at every entry point, because construction-time validation makes invalid values unrepresentable and gave [related behaviour] a natural home."

Replace Array with Object (Fowler 186)

Definition. "You have an array in which certain elements mean different things. Replace the array with an object that has a field for each element" (Refactoring1 p.26).

Before:

```
String[] row = new String[2];
row[0] = "Liverpool";  // name, by convention
row[1] = "15";         // wins, by convention
```

After:

```
class Performance {
    private String name;
    private int wins;
    // getters/setters with real names and types
}
```

When/why. When an array is abused as a record — positions carry meaning only by convention, which the compiler cannot check. The object gives named, typed fields. **Mechanics:** create the class with a field per element; replace array reads/writes with accessors one element at a time; remove the array [Fowler99]. **Reflection snippet.** "I replaced the convention-indexed array in [Class] with a [Record] object rather than documenting what each index means, because named typed fields turn silent positional mistakes into compile errors."

Duplicate Observed Data (Fowler 189)

Definition. "You have domain data available only in a GUI control, and domain methods need access. Copy the data to a domain object. Set up an observer to synchronize the two pieces of data" (Refactoring1 p.27).

Before:

```
class IntervalWindow extends Frame {
    TextField startField, endField, lengthField;
    void calculateLength() { /* parses the widgets directly */ }
}
```

After:

```
class Interval extends Observable {   // domain object owns the data
    private String start, end, length;
}
class IntervalWindow implements Observer {
    public void update(Observable o, Object arg) { /* refresh widgets */ }
}
```

When/why. The first move when separating domain logic from the UI — domain data moves to a domain object, and the Observer pattern keeps the widget in sync. Directly relevant to JHotDraw's MVC split and the big refactoring Separate Domain from Presentation. **Mechanics:** make the window an observer of a new domain class; move each piece of data into the domain object, replacing widget access with accessor calls plus update notifications; test after each piece [Fowler99]. **Reflection snippet.** "I duplicated the observed data from [Window] into a domain class [Model] with an Observer link, instead of letting calculation code parse text fields, because domain logic that lives in widgets cannot be unit tested — afterwards [Model] was testable headlessly."

Encapsulate Field (Fowler 206)

Definition. "There is a public field. Make it private and provide accessors" (Refactoring1 p.28).

Before:

```
public String name;
```

After:

```
private String name;
public String getName() { return name; }
public void setName(String name) { this.name = name; }
```

When/why. The basic encapsulation move — a public field lets any code mutate state without the class knowing; accessors give the class control and let representation change later. **Mechanics:** make the field private; create accessors; redirect all external users; compile and test [Fowler99]. **Reflection snippet.** "I made [field] private behind accessors instead of leaving it public, because every external writer was a hidden coupling to the representation; with the setter in place I could later add [validation/notification] in exactly one place."

Encapsulate Collection (Fowler 208)

Definition. "A method returns a collection. Make it return a read-only view and provide add/remove methods" (Refactoring1 p.29).

Before:

```
class Course { }
class Person {
    private Set<Course> courses;
    Set<Course> getCourses() { return courses; }       // live reference!
    void setCourses(Set<Course> c) { courses = c; }
}
```

After:

```
class Person {
    private Set<Course> courses = new HashSet<>();
    Set<Course> getCourses() { return Collections.unmodifiableSet(courses); }
    void addCourse(Course c) { courses.add(c); }
    void removeCourse(Course c) { courses.remove(c); }
}
```

When/why. A getter handing out a live collection lets callers mutate the owner's state behind its back; the read-only view plus add/remove methods restores the single guardian of the class's invariants. **Mechanics:** add add/remove methods; initialise the field; replace external mutations with the new methods; make the getter return an unmodifiable view; remove the bulk setter [Fowler99]. **Reflection snippet.** "I changed [getCollection] to return an unmodifiable view and added [add/remove] methods, instead of trusting every caller not to mutate the set, because the class can only maintain its invariants if all changes flow through it."

Replace Subclass with Fields (Fowler 232)

Definition. "You have subclasses that vary only in methods that return constant data. Change the methods to superclass fields and eliminate the subclasses" (Refactoring1 p.30).

Before:

```
abstract class Person { abstract boolean isMale(); abstract char getCode(); }
class Male extends Person   { boolean isMale() { return true; }  char getCode() { return 'M'; } }
class Female extends Person { boolean isMale() { return false; } char getCode() { return 'F'; } }
```

After:

```
class Person {
    private final boolean male; private final char code;
    private Person(boolean male, char code) { this.male = male; this.code = code; }
    static Person createMale()   { return new Person(true, 'M'); }
    static Person createFemale() { return new Person(false, 'F'); }
}
```

When/why. Subclasses that differ only by constant return values are pulling their weight as data, not behaviour — collapse them. Fixes Lazy Class and over-use of inheritance. **Mechanics:** add a field per constant method; add a protected constructor setting them; create factory methods per former subclass; replace the constant methods with field reads; remove the subclasses [Fowler99]. **Reflection snippet.** "The

[Sub] classes varied only in constant values, so I collapsed them into fields with factory methods rather than keeping a class file per constant — the hierarchy disappeared and the variants became data."

Decompose Conditional (Fowler 238)

Definition. "You have a complicated conditional (if-then-else) statement. Extract methods from the condition, then part, and else parts" (Refactoring1 p.32).

Before:

```
if (date.before(SUMMER_START) || date.after(SUMMER_END))
    charge = quantity * winterRate + winterServiceCharge;
else
    charge = quantity * summerRate;
```

After:

```
if (notSummer(date)) charge = winterCharge(quantity);
else                 charge = summerCharge(quantity);
```

When/why. When the intent of a conditional is buried in long boolean expressions and branch bodies — naming each part makes the logic read like prose and shrinks branch-point density toward the G2 limit (BetterCode p.13). **Mechanics:** Extract Method on the condition; Extract Method on the then-branch; Extract Method on the else-branch; test after each [Fowler99]. **Reflection snippet.** "I decomposed the conditional in [method] into [notSummer]/[winterCharge]/[summerCharge]-style named parts instead of commenting each branch, because the names carry the domain meaning and each piece became separately testable."

Consolidate Conditional Expression (Fowler 240)

Definition. "You have a sequence of conditional tests with the same result. Combine them into a single conditional expression and extract it" (Refactoring1 p.33).

Before:

```
double disabilityAmount() {
    if (seniority < 2) return 0;
    if (monthsDisabled > 12) return 0;
    if (isPartTime) return 0;
    // compute amount...
}
```

After:

```
double disabilityAmount() {
    if (isNotEligibleForDisability()) return 0;
    // compute amount...
}
```

When/why. Several separate checks leading to the same outcome are really one question; merging and naming them tells the reader "these are all the same case" and creates a reusable predicate. **Mechanics:** verify the conditions have no side effects; combine with and/or; Extract Method on the combined condition; test [Fowler99]. **Reflection snippet.** "I consolidated the three early-return checks in [method] into one

named predicate [isNotEligible], instead of leaving the reader to discover they share a result, because one named question reads faster than three anonymous ones."

Replace Nested Conditional with Guard Clauses (Fowler 250)

Definition. "A method has conditional behavior that does not make clear the normal path of execution. Use guard clauses for all the special cases" (Refactoring1 p.34).

Before:

```
double getPayAmount() {
    double result;
    if (isDead) result = deadAmount();
    else {
        if (isSeparated) result = separatedAmount();
        else {
            if (isRetired) result = retiredAmount();
            else result = normalPayAmount();
        }
    }
    return result;
}
```

After:

```
double getPayAmount() {
    if (isDead) return deadAmount();
    if (isSeparated) return separatedAmount();
    if (isRetired) return retiredAmount();
    return normalPayAmount();
}
```

When/why. Deep nesting hides the mainline; guard clauses give each special case an early exit so the normal path is the un-indented body. **Mechanics:** take the outermost special case and convert it to a guard clause with an immediate return; repeat inward; test after each [Fowler99]. **Reflection snippet.** "I flattened the nested conditional in [method] into guard clauses instead of keeping one exit point, because the single-exit rule was costing three levels of indentation; with the special cases returning early, the normal path is now the visible spine of the method."

Replace Conditional with Polymorphism (Fowler 255)

Definition. "You have a conditional that chooses different behavior depending on the type of an object. Move each leg of the conditional to an overriding method in a subclass. Make the original method abstract" (Refactoring1 p.35). Also BetterCode G2's recommended refactoring (BetterCode p.18).

Before:

```
double getSpeed() {
    switch (type) {
        case EUROPEAN: return getBaseSpeed();
        case AFRICAN:  return getBaseSpeed() - getLoadFactor() * numCoconuts;
        case NORWEGIAN_BLUE: return isNailed ? 0 : getBaseSpeed(voltage);
    }
    throw new RuntimeException("unreachable");
}
```

After:

```
abstract class Bird { abstract double getSpeed(); }
class European extends Bird { double getSpeed() { return getBaseSpeed(); } }
class African extends Bird {
    double getSpeed() { return getBaseSpeed() - getLoadFactor() * numCoconuts; }
}
class NorwegianBlue extends Bird {
    double getSpeed() { return isNailed ? 0 : getBaseSpeed(voltage); }
}
```

When/why. The canonical fix for the Switch Statements smell: each subclass owns its case, so adding a new type means adding a class, not editing every switch — the Open/Closed Principle in action (DesignPrinciplesAndPatterns p.5). **Mechanics:** ensure a hierarchy exists (Replace Type Code with Subclasses first if not); push one leg at a time into the matching subclass override; when all legs are gone, make the superclass method abstract; test after each leg [Fowler99]. **Reflection snippet.** "I replaced the type switch on [typeCode] with polymorphism instead of adding yet another case, because the same switch existed in [N] methods and every new [type] meant finding them all; now a new [type] is one new subclass that the compiler forces to implement every operation."

Introduce Null Object (Fowler 260)

Definition. "You have repeated checks for a null value. Replace the null value with a null object" (Refactoring1 p.36). Cross-listed in Kerievsky as a pattern-level refactoring (HighLevelRefactoring p.12).

Before:

```
Customer c = site.getCustomer();
String name = (c == null) ? "occupant" : c.getName();
BillingPlan plan = (c == null) ? BillingPlan.basic() : c.getPlan();
```

After:

```
class NullCustomer extends Customer {
    String getName() { return "occupant"; }
    BillingPlan getPlan() { return BillingPlan.basic(); }
}
// site.getCustomer() now returns a NullCustomer instead of null:
String name = site.getCustomer().getName();
```

When/why. When `if (x != null)` checks litter the code, an object implementing the neutral behaviour makes them all disappear — the default behaviour gets one authoritative home. **Mechanics:** create the null subclass with an isNull-style marker; make providers return it instead of null; replace each null check with plain calls, moving the default behaviour into the null class; test after each [Fowler99]. **Reflection snippet.** "I introduced a [NullThing] object instead of repeating null checks in [N] call sites, because the 'no [thing]' behaviour was duplicated policy — as a class it is defined once, and a forgotten null check can no longer crash [feature]."

Separate Query from Modifier (Fowler 279)

Definition. "You have a method that returns a value but also changes the state of an object. Create two methods, one for the query and one for the modification" (Refactoring1 p.38).

Before:

```
String foundMiscreant(String[] people) {
    // returns the name AND sends an alert (side effect)
}
```

After:

```
String foundPerson(String[] people) { /* pure query */ }
void sendAlert(String[] people)     { /* the modifier  */ }
```

When/why. Enforces Command-Query Separation (CleanCode p.23–33 area): a value-returning method with side effects is surprising and unsafe to call repeatedly; after the split, queries can be called freely and the modifier's effect is explicit. **Mechanics:** create a side-effect-free query returning the same value; adjust each caller to call query (and modifier where needed); strip the return from the original modifier; test [Fowler99]. **Reflection snippet.** "I split [method] into a pure query [getX] and a modifier [doY] instead of leaving a getter with a hidden side effect, because callers were re-invoking it 'just to read' and unknowingly [side effect]; after the split the test for the query needs no teardown."

Parameterize Method (Fowler 283)

Definition. "Several methods do similar things but with different values contained in the method body. Create one method that uses a parameter for the different values" (Refactoring1 p.39).

Before:

```
void tenPercentRaise()  { salary *= 1.10; }
void fivePercentRaise() { salary *= 1.05; }
```

After:

```
void raise(double factor) { salary *= (1 + factor); }
```

When/why. Near-identical methods that differ only by a literal are a flavour of Duplicated Code — one parameterised method removes the family. **Mechanics:** create the parameterised method; convert one duplicate at a time into a call to it; test after each; delete the duplicates [Fowler99]. **Reflection snippet.** "I folded [methodA] and [methodB] into one parameterised [method], instead of adding a third sibling for the new case, because the family differed only in the literal [value] — the next variant is now a call, not a copy."

Replace Parameter with Method (Fowler 292)

Definition. "An object invokes a method, then passes the result as a parameter for a method. The receiver can also invoke this method. Remove the parameter and let the receiver invoke the method" (Refactoring1 p.40).

Before:

```
int basePrice = quantity * itemPrice;
double discountLevel = getDiscountLevel();
double finalPrice = discountedPrice(basePrice, discountLevel);
```

After:

```
int basePrice = quantity * itemPrice;
double finalPrice = discountedPrice(basePrice);
// discountedPrice now calls getDiscountLevel() itself
```

When/why. If the receiver can compute a parameter's value itself, passing it is noise — shorter signatures serve G4 (BetterCode p.25). Not applicable if the computation depends on caller-local state. **Mechanics:** extract the parameter's computation into a method if needed; replace parameter references in the body with calls to that method; Remove Parameter; test [Fowler99]. **Reflection snippet.** "I removed the [param] parameter from [method] and let the method fetch the value itself, instead of threading it through every caller, because each caller computed it identically — the signature shrank and a caller can no longer pass an inconsistent value."

Introduce Parameter Object (Fowler 295)

Definition. "You have a group of parameters that naturally go together. Replace them with an object" (Refactoring1 p.41). The named fix for SIG G4 "extract parameters into objects" (BetterCode p.25) and for Data Clumps.

Before:

```
List<Entry> getFlow(Date start, Date end, double minAmount, double maxAmount)
```

After:

```
List<Entry> getFlow(DateRange range, AmountRange amounts)
class DateRange {
    private final Date start, end;
    boolean includes(Date d) { return !d.before(start) && !d.after(end); }
}
```

When/why. A recurring parameter cluster is a missing concept; wrapping it shortens every signature and gives related behaviour (like `includes`) a home. **Mechanics:** create an immutable parameter class; add it to the signature alongside the old parameters; migrate callers; replace uses of the old parameters with the object's accessors; drop the old parameters; then look for behaviour to move in [Fowler99]. **Reflection snippet.** "I bundled [params] into a [ParamObject] instead of leaving a [N]-parameter signature, because the values always travelled together; the signature now meets G4's at-most-4 rule, and the new class immediately absorbed the [validation/range] logic that had been duplicated in callers."

Pull Up Constructor Body (Fowler 325)

Definition. "You have constructors on subclasses with mostly identical bodies. Create a superclass constructor; call this from the subclass methods" (Refactoring1 p.43).

Before:

```
class Manager extends Employee {
    Manager(String name, String id, int grade) {
        this.name = name; this.id = id;   // duplicated in every sibling
        this.grade = grade;
    }
}
```

After:

```
class Employee {
    protected Employee(String name, String id) { this.name = name; this.id = id; }
}
class Manager extends Employee {
    Manager(String name, String id, int grade) {
        super(name, id);
        this.grade = grade;
    }
}
```

When/why. Constructor duplication across siblings is Duplicated Code in the one place ordinary Pull Up Method cannot reach (constructors are not inherited). **Mechanics:** define the superclass constructor with the common assignments; call it via super as the first statement in each subclass constructor; test [Fowler99]. **Reflection snippet.** "I pulled the shared constructor body of [SubA] and [SubB] up into [Super], instead of copying the initialisation into the new third subclass, so common setup now exists once and a new sibling starts with `super(...)`."

Extract Subclass (Fowler 330)

Definition. "A class has features that are used only in some instances. Create a subclass for that subset of features" (Refactoring1 p.44).

Before:

```
class JobItem {
    int unitPrice; Employee employee; boolean isLabor;  // employee only when isLabor
    int getUnitPrice() { return isLabor ? employee.getRate() : unitPrice; }
}
```

After:

```
class JobItem { int getUnitPrice() { return unitPrice; } }
class LaborItem extends JobItem {
    Employee employee;
    int getUnitPrice() { return employee.getRate(); }
}
```

When/why. When some fields/methods serve only a subset of instances — often flagged by a type code or Temporary Field — split that variant into a subclass and the flag disappears into the type system. **Mechanics:** create the subclass; provide constructors/factories; replace creation sites of the variant; push down the variant-only features; eliminate the discriminating flag; test [Fowler99]. **Reflection snippet.** "I extracted [VariantSub] from [Class] instead of keeping the `isX` flag and its conditionals, because the flag was steering behaviour in [N] methods; the subclass made each conditional an override and the impossible field combinations unrepresentable."

Extract Superclass (Fowler 336)

Definition. "You have two classes with similar features. Create a superclass and move the common features to the superclass" (Refactoring1 p.45).

Before:

```
class Department { String name; List<Employee> staff; int totalAnnualCost() { /*...*/ } }
class Employee   { String name; int id; int annualCost; int totalAnnualCost() { /*...*/ } }
```

After:

```
abstract class Party {
    protected String name;
    abstract int totalAnnualCost();
    String getName() { return name; }
}
class Department extends Party { /* ... */ }
class Employee   extends Party { /* ... */ }
```

When/why. Two siblings sharing fields and behaviour are Duplicated Code across classes; a common parent stores the commonality once and lets clients treat both polymorphically. **Mechanics:** create an (abstract) superclass; Pull Up Field, Pull Up Method, Pull Up Constructor Body one element at a time; make divergent same-intent methods abstract in the parent; test throughout [Fowler99]. **Reflection snippet.** "I lifted the duplicated [feature] of [ClassA] and [ClassB] into a new superclass [Parent], instead of letting the copies drift apart, because the two implementations had already diverged once in a bug; the shared parent makes future divergence impossible."

Extract Interface (Fowler 341)

Definition. "Several clients use the same subset of a class's interface, or two classes have part of their interfaces in common. Extract the subset into an interface" (Refactoring1 p.46).

Before:

```
class Employee {
    int getRate() { /* ... */ }
    boolean hasSpecialSkill() { /* ... */ }
    void promote() { /* ... */ }   // clients of billing never use this
}
double charge(Employee emp, int days) { return emp.getRate() * days; }
```

After:

```
interface Billable { int getRate(); boolean hasSpecialSkill(); }
class Employee implements Billable { /* ... */ }
double charge(Billable emp, int days) { return emp.getRate() * days; }
```

When/why. When clients depend only on a slice of a class, capturing the slice as an interface makes the dependency explicit and minimal — this is the Interface Segregation Principle made mechanical (DesignPrinciplesAndPatterns p.15) and the precondition for substituting test doubles. **Mechanics:** create the interface; declare the needed methods; make the class implement it; retype clients to the interface; test [Fowler99]. **Reflection snippet.** "I extracted [Interface] from [Class] and retyped [Client] to it, instead of letting [Client] see the whole class, because [Client] used only [n] of [m] methods; the narrowed dependency also let me substitute a mock of [Interface] in the unit tests."

Collapse Hierarchy (Fowler 344)

Definition. "A superclass and subclass are not very different. Merge them together" (Refactoring1 p.46).

Before:

```
class Employee { /* almost everything */ }
class Salesman extends Employee { /* one trivial override left */ }
```

After:

```
class Employee { /* everything, flag or field covers the difference */ }
```

When/why. After refactorings drain a subclass (or parent), the remaining distinction may not justify a class — merging removes a level of indirection readers must traverse. Fixes Lazy Class in a hierarchy. **Mechanics:** Pull Up / Push Down the remaining features; redirect references to the surviving class; delete the empty class; test [Fowler99]. **Reflection snippet.** "After moving [behaviour] out, [Sub] differed from [Super] by one field, so I collapsed the hierarchy rather than preserving an inheritance level whose only content was historical."

Form Template Method (Fowler 345)

Definition. "You have two methods in subclasses that perform similar steps in the same order, yet the steps are different. Get the steps into methods with the same signature, so that the original methods become the same. Then you can pull them up" (Refactoring1 p.47). Implements the Template Method design pattern; Kerievsky lists it as a Duplicated Code fix (HighLevelRefactoring p.12).

Before:

```
class ResidentialSite extends Site {
    double getBillableAmount() { /* base + tax, residential formulas */ }
}
class LifelineSite extends Site {
    double getBillableAmount() { /* base + tax, lifeline formulas */ }
}
```

After:

```
abstract class Site {
    double getBillableAmount() { return getBaseAmount() + getTaxAmount(); }
    abstract double getBaseAmount();
    abstract double getTaxAmount();
}
```

When/why. Sibling methods sharing an algorithm skeleton but differing in steps: the skeleton moves up as the template; the varying steps stay down as overridable hooks. Kills structural duplication that plain Extract Method cannot reach across classes. **Mechanics:** decompose each sibling method so the varying parts become same-signature methods; pull the now-identical skeleton up; leave the hooks abstract; test [Fowler99]. **Reflection snippet.** "The [operation] in [SubA] and [SubB] followed the same three steps with different formulas, so I formed a template method in [Super] with abstract hooks, instead of extracting a shared helper with flags — the skeleton is now stated once and each subclass owns only its genuinely different steps."

Replace Inheritance with Delegation (Fowler 352)

Definition. "A subclass uses only part of a superclass's interface or does not want to inherit data. Create a field for the superclass, adjust methods to delegate to the superclass, and remove the subclassing" (Refactoring1 p.48).

Before:

```
class MyStack extends Vector<Object> {
    void push(Object o) { insertElementAt(o, 0); }
    Object pop() { /* ... */ }
    // also exposes all 45 Vector methods, breaking the stack discipline
}
```

After:

```
class MyStack {
    private final Vector<Object> vector = new Vector<>();
    void push(Object o) { vector.insertElementAt(o, 0); }
    Object pop() { Object r = vector.firstElement(); vector.removeElementAt(0); return r; }
    int size() { return vector.size(); }
}
```

When/why. The cure for Refused Bequest — when "is-a" is false, composition expresses "uses-a" honestly, and the class's public surface shrinks to what it really supports (composition over inheritance, CRP). **Mechanics:** add a field referencing the former parent (initially `this`); change methods to delegate through the field; remove the extends; create the delegate; add forwarding methods for what clients need; test [Fowler99]. **Reflection snippet.** "[Sub] inherited [Super] but honoured only part of its contract, so I replaced the inheritance with a delegate field, instead of overriding unwanted methods to throw — clients now see only the [n] operations [Sub] truly supports, and Liskov substitution is no longer violated."

Replace Delegation with Inheritance (Fowler 355)

Definition. "You're using delegation and are often writing many simple delegations for the entire interface. Make the delegating class a subclass of the delegate" (Refactoring1 p.49). Inverse of the previous.

Before:

```
class Employee {
    private Person person = new Person();
    String getName() { return person.getName(); }
    void setName(String n) { person.setName(n); }
    String toString() { return "Emp: " + person.getLastName(); }
    // forwards person's entire interface
}
```

After:

```
class Employee extends Person {
    String toString() { return "Emp: " + getLastName(); }
}
```

When/why. When a class delegates the delegate's *whole* interface and the "is-a" genuinely holds, inheritance deletes the boilerplate. Only when all of it is used — otherwise you would be manufacturing Refused Bequest.

Mechanics: make the delegator a subclass; point the delegate field at `this`; remove the forwarding methods one by one; drop the field; test [Fowler99]. **Reflection snippet.** "[Wrapper] forwarded every single method of [Inner] unchanged and the is-a test held, so I made it a subclass instead of maintaining [N] one-line forwards that had to grow with every new method."

Tease Apart Inheritance (big refactoring)

Definition. "You have an inheritance hierarchy that is doing two jobs at once. Create two hierarchies and use delegation to invoke one from the other" (Refactoring1 p.52).

Before:

```
// one hierarchy tangles two dimensions:
class Deal { }
class ActiveDeal extends Deal { }
class PassiveDeal extends Deal { }
class TabularActiveDeal extends ActiveDeal { }   // presentation x kind!
class TabularPassiveDeal extends PassiveDeal { }
```

After:

```
class Deal { private PresentationStyle style; }   // delegation bridges the two
class ActiveDeal extends Deal { }
class PassiveDeal extends Deal { }
class PresentationStyle { }
class TabularPresentation extends PresentationStyle { }
```

When/why. When subclass names contain two adjectives (the tell), the hierarchy multiplies combinations and fixes Parallel Inheritance Hierarchies at the root; afterwards each dimension varies independently — structurally this is the Bridge pattern (DesignPrinciplesAndPatterns p.31). **Mechanics:** identify the two jobs; create a new hierarchy for the secondary job; give the original a delegate field to it; migrate the duplicated combinations downward into delegation; collapse leftovers [Fowler99]. **Reflection snippet.** "Our hierarchy multiplied [dimension A] by [dimension B] into [N] leaf classes; I teased it apart into two hierarchies joined by delegation, instead of adding the [N+1]th combination class, so a new [A] or a new [B] is now one class instead of a row or column of them."

Convert Procedural Design to Objects (big refactoring)

Definition. "You have code written in a procedural style. Turn the data records into objects, break up the behavior, and move the behavior to the objects" (Refactoring1 p.54).

Before:

```
class OrderCalculator {     // one fat "program" class
    static double calc(OrderRecord r) { /* 200 lines using r's fields */ }
}
class OrderRecord { public double[] amounts; public int type; }
```

After:

```
class Order {
    private List<Money> amounts; private OrderType type;
```

```
    double total() { /* behaviour lives with the data */ }
}
```

When/why. Legacy procedural code — dumb records plus god-functions — resists change because behaviour and data are apart; moving logic onto the records yields testable, cohesive objects. **Mechanics:** turn each record into a class with accessors; Extract Method on the procedural body; Move Method each piece onto the record it concerns; repeat until the original "program" class is thin or gone [Fowler99]. **Reflection snippet.** "[Module] was a record class plus a static calculator; I converted it to objects by extracting and moving each behaviour onto [Record], instead of patching the 200-line procedure again, because every change request was landing in the same untestable function."

Separate Domain from Presentation (big refactoring)

Definition. "You have GUI classes that contain domain logic. Separate the domain logic into separate domain classes" (Refactoring1 p.56). The MVC split as a refactoring; directly relevant to JHotDraw's GUI/domain layering.

Before:

```
class OrderWindow extends JFrame {
    void onCalculateClick() {
        double total = 0;   // pricing rules live in the widget handler
        for (Row r : table.rows()) total += r.qty * r.price * (r.vip ? 0.9 : 1.0);
        totalField.setText(String.valueOf(total));
    }
}
```

After:

```
class Order { double total() { /* pricing rules, no Swing imports */ } }
class OrderWindow extends JFrame {
    void onCalculateClick() { totalField.setText(String.valueOf(order.total())); }
}
```

When/why. Domain logic in widgets cannot be unit tested, cannot be reused in a second UI, and changes for two reasons (look and rules). The split gives a headless, testable domain — Duplicate Observed Data is the per-datum move inside this big one. **Mechanics:** create domain classes per window concept; use Duplicate Observed Data to migrate state; Move Method the logic across; leave the window as a thin observer/controller [Fowler99]. **Reflection snippet.** "I moved the [pricing/validation] rules out of [Window] into a domain class [Domain], instead of testing through the GUI, because under-the-GUI logic needed AssertJ-Swing robots to verify; afterwards the same rules were five plain JUnit tests."

Extract Hierarchy (big refactoring)

Definition. "You have a class that is doing too much work, at least in part through many conditional statements. Create a hierarchy of classes in which each subclass represents a special case" (Refactoring1 p.58). The whole-class cousin of Replace Conditional with Polymorphism.

Before:

```
class BillingScheme {
    double charge(...) {
        if (isBusiness()) { /* ... */ }
        else if (isResidential()) { /* ... */ }
```

```
        else if (isDisability()) { /* ... */ }
        // each method repeats this fan-out
    }
}
```

After:

```
abstract class BillingScheme { abstract double charge(/* ... */); }
class BusinessBillingScheme    extends BillingScheme { /* ... */ }
class ResidentialBillingScheme extends BillingScheme { /* ... */ }
class DisabilityBillingScheme  extends BillingScheme { /* ... */ }
```

When/why. One class handling many variants through repeated conditional fan-outs: grow a subclass per case so each variant's logic is collected in one place and new cases are additive. **Mechanics:** create a subclass for one special case; use a factory to instantiate it where the conditions identify that case; push the case's logic down one method at a time; repeat per case; leave the parent abstract [Fowler99]. **Reflection snippet.** " [Class] dispatched on [condition] in [N] different methods; I extracted a hierarchy with one subclass per case, instead of decomposing each conditional locally, because the same fan-out repeated everywhere — one subclass now collects everything about [case] that was previously sliced across the class."

Kerievsky's composite refactorings — the pattern-level catalog at a glance

Kerievsky's "Refactoring to Patterns" treats design patterns as *targets* of composite refactorings (HighLevelRefactoring p.12–13). The smell-to-composite table, paste-ready:

- Duplicated Code → Form Template Method (205); Introduce Polymorphic Creation with Factory Method (88); Chain Constructors (340); Replace One/Many Distinctions with Composite (224); Extract Composite (214); Unify Interfaces with Adapter (247); Introduce Null Object (301).
- Long Method → Compose Method (123); Move Accumulation to Collecting Parameter (313); Replace Conditional Dispatcher with Command (191); Move Accumulation to Visitor (320); Replace Conditional Logic with Strategy (129).
- Conditional Complexity → Replace Conditional Logic with Strategy (129); Move Embellishment to Decorator (144); Replace State-Altering Conditionals with State (166); Introduce Null Object (301).
- Primitive Obsession → Replace Type Code with Class (286); Replace State-Altering Conditionals with State (166); Replace Conditional Logic with Strategy (129); Replace Implicit Tree with Composite (178); Encapsulate Composite with Builder (96); Move Embellishment to Decorator (144); Replace Implicit Language with Interpreter (269).
- Switch Statements → Replace Conditional Dispatcher with Command (191); Move Accumulation to Visitor (320).
- Large Class → Replace Conditional Dispatcher with Command (191); Replace State-Altering Conditionals with State (166); Replace Implicit Language with Interpreter (269).
- Lazy Class → Inline Singleton (114). Alternative Classes with Different Interfaces → Unify Interfaces with Adapter (247). Combinatorial Explosion → Replace Implicit Language with Interpreter (269). Indecent Exposure → Encapsulate Classes with Factory (80). Oddball Solution → Unify Interfaces with Adapter (247). Solution Sprawl → Move Creation Knowledge to Factory (68).

Duplicated Code is the most-served smell (seven distinct fixes); the three big conditional refactorings (Strategy, State, Command) each serve three smells (HighLevelRefactoring p.12–13).

Replace Conditional Logic with Strategy (Kerievsky 129) — worked example

Definition. Move each variant of a conditional calculation into its own Strategy class and delegate to it; the host holds a Strategy reference (HighLevelRefactoring p.12–13; resolves Conditional Complexity, Long Method, Primitive Obsession).

Before:

```
class Loan {
    double capital() {
        if (expiry == null && maturity != null) return /* term loan formula */;
        if (expiry != null && maturity == null) {
            if (getUnusedPercentage() != 1.0) return /* revolver formula A */;
            else return /* revolver formula B */;
        }
        return 0.0;
    }
}
```

After:

```
interface CapitalStrategy { double capital(Loan loan); }
class TermLoanCapital implements CapitalStrategy { public double capital(Loan l) { /* ... */ } }
class RevolverCapital implements CapitalStrategy { public double capital(Loan l) { /* ... */ } }
class Loan {
    private final CapitalStrategy strategy;
    double capital() { return strategy.capital(this); }
}
```

When/why. The conditional encodes a family of interchangeable algorithms; as Strategies, each is small, separately testable, and new variants are added without touching the host (OCP). Prefer over plain Replace Conditional with Polymorphism when the host class cannot or should not be subclassed (the variation is a property, not an identity). **Reflection snippet.** "I moved the [calculation] variants behind a [Strategy] interface instead of subclassing [Host] per variant, because [Host]'s identity does not change with the algorithm — and a Strategy can be swapped at runtime and mocked in tests."

Replace Conditional Dispatcher with Command (Kerievsky 191) — worked example

Definition. Replace a switch-based request dispatcher with a map of Command objects sharing an execute interface (HighLevelRefactoring p.12–13; resolves Large Class, Long Method, Switch Statements).

Before:

```
void handle(String action) {
    if (action.equals("new"))       { /* 15 lines */ }
    else if (action.equals("open")) { /* 20 lines */ }
    else if (action.equals("save")) { /* 25 lines */ }
    // grows with every new action
}
```

After:

```
interface Command { void execute(); }
Map<String, Command> commands = Map.of(
    "new",  new NewFileCommand(editor),
    "open", new OpenFileCommand(editor),
```

```
    "save", new SaveFileCommand(editor));
void handle(String action) { commands.get(action).execute(); }
```

When/why. The dispatcher stops growing: each action is a class with its own state, tests, and — as in JHotDraw's command-based menu structure — a natural place for undo support. Adding an action means writing a class and one map entry. **Reflection snippet.** "I replaced the action dispatcher's if-chain with Command objects in a map, instead of appending an [N+1]th branch, because JHotDraw's own menus use Command — each action became independently testable and gained a hook for undo."

Replace State-Altering Conditionals with State (Kerievsky 166) — worked example

Definition. When conditionals inside a class change its mode and steer behaviour by that mode, move each mode into a State class; the host delegates to its current state object, and the states perform the transitions (HighLevelRefactoring p.12–13; resolves Conditional Complexity, Large Class, Primitive Obsession).

Before:

```
class Permission {
    private int state = REQUESTED;          // type-code mode field
    void claim()  { if (state == REQUESTED) { state = CLAIMED; /* ... */ } }
    void grant()  { if (state == CLAIMED && isValid()) { state = GRANTED; /* ... */ } }
    void deny()   { if (state == CLAIMED) { state = DENIED; /* ... */ } }
}
```

After:

```
abstract class PermissionState {
    void claim(Permission p) {}             // default: ignore
    void grant(Permission p) {}
}
class Requested extends PermissionState {
    void claim(Permission p) { p.setState(new Claimed()); /* ... */ }
}
class Claimed extends PermissionState {
    void grant(Permission p) { if (p.isValid()) p.setState(new Granted()); }
}
class Permission {
    private PermissionState state = new Requested();
    void claim() { state.claim(this); }
    void grant() { state.grant(this); }
}
```

When/why. The mode field plus its guards is a state machine written as scattered conditionals; as State classes, each mode's legal transitions are collected in one place, illegal transitions become no-ops or errors by design, and a new mode is a new class. **Reflection snippet.** "[Class] tracked its mode in an int and guarded every method with mode checks; I introduced the State pattern so each mode class owns its behaviour and transitions — the [N] scattered conditionals collapsed into delegation, and the state diagram is now readable directly from the class list."

Move Embellishment to Decorator (Kerievsky 144) — worked example

Definition. When optional extra behaviour (an embellishment) has been wired into a class behind condition flags, move it into a Decorator that wraps the plain class (HighLevelRefactoring p.12–13; resolves Conditional Complexity, Primitive Obsession).

Before:

```
class Invoice {
    private boolean withReminderText;        // optional embellishment flag
    String render() {
        String s = renderCore();
        if (withReminderText) s += renderReminder();   // and more flags follow
        return s;
    }
}
```

After:

```
interface InvoiceRenderer { String render(); }
class PlainInvoice implements InvoiceRenderer {
    public String render() { return renderCore(); }
}
class ReminderDecorator implements InvoiceRenderer {
    private final InvoiceRenderer inner;
    ReminderDecorator(InvoiceRenderer inner) { this.inner = inner; }
    public String render() { return inner.render() + renderReminder(); }
}
// composition at creation time:
InvoiceRenderer r = new ReminderDecorator(new PlainInvoice());
```

When/why. Each flag multiplies the conditional paths through the host; decorators make embellishments composable wrappers, so combinations are built by stacking rather than by branching — JHotDraw uses exactly this shape in its figure decorators (e.g. a border decorator wrapping a figure). **Reflection snippet.** "The optional [embellishment] lived behind a flag in [Class], and a second option was about to double the paths; I moved it to a Decorator wrapping the plain class — mirroring JHotDraw's own figure decorators — so options now compose by stacking wrappers instead of multiplying conditionals."

Compose Method (Kerievsky 123) — worked example

Definition. Transform a method's body into a short sequence of same-level steps, each a call to a well-named intention-revealing method (HighLevelRefactoring p.12–13; resolves Long Method). It is Extract Method applied until the residue reads as prose.

Before:

```
void add(Object element) {
    if (!readOnly) {
        int newSize = size + 1;
        if (newSize > elements.length) {
            Object[] newElements = new Object[elements.length + 10];
            for (int i = 0; i < size; i++) newElements[i] = elements[i];
            elements = newElements;
        }
        elements[size++] = element;
    }
}
```

After:

```
void add(Object element) {
    if (readOnly) return;
    if (atCapacity()) grow();
```

```
    addElement(element);
}
```

When/why. The composed method states the policy in three lines at one abstraction level; the mechanics live one level down in named helpers. This is the Stepdown Rule (CleanCode p.24) achieved through refactoring, and the standard route to satisfying G1 and G2 simultaneously. **Reflection snippet.** "I composed [method] into a guard clause plus [N] intention-named steps instead of leaving the mixed-level body, because the composed version can be read at a glance and each step tested alone — the method now is its own summary."

Pipeline deep dive — build it, explain it, justify it

How did you create your pipeline — the narrative answer

Paste-ready first-person narrative (adapt the bracketed parts):

I created my CI pipeline with GitHub Actions, following the CILab's five classwork steps (CILab p.1). First I read GitHub's guide "Building and testing Java with Maven" to adopt the documented Actions-plus-Maven recipe. Second, I added a workflow file `maven.yml` under `.github/workflows/` in my JHotDraw fork — the path GitHub Actions scans for pipeline definitions — so the pipeline definition itself is version-controlled in the same repository as the code it builds. Third, I configured the workflow to trigger automatically on every pull request (and on pushes to my main branch), so each proposed integration is built with Maven before it can merge — the practical realisation of "commit frequently and build every commit" (ContinuousIntegration p.3, p.5). Fourth, because the lab uses shared jars from GitHub Packages, I added a `.maven-settings.xml` in the project root so the build server resolves dependencies from the registry rather than from anyone's laptop (CILab p.1). Fifth, I configured the workflow to execute the test suite automatically via Maven's test phase, making the build self-testing (ContinuousIntegration p.9): a green run means working software, not merely compiling software. I verified the pipeline by pushing a deliberately failing test and confirming the run went red, then fixing it and confirming green — evidence that the safety net actually catches failures.

The complete GitHub Actions workflow YAML

(beyond slides — practical knowledge: the deck teaches the principles, the lab names the tools; this concrete YAML is standard GitHub Actions practice for a Maven JHotDraw-style project.)

```
name: Java CI with Maven

on:
  push:
    branches: [ "main" ]
  pull_request:
    branches: [ "main" ]

jobs:
  build:
    runs-on: ubuntu-latest

    steps:
      - name: Check out repository
        uses: actions/checkout@v4

      - name: Set up JDK 11
        uses: actions/setup-java@v4
        with:
```

```
          java-version: '11'
          distribution: 'temurin'
          cache: maven

      - name: Build and run tests
        run: mvn --batch-mode --update-snapshots verify

      - name: Upload test reports
        if: always()
        uses: actions/upload-artifact@v4
        with:
          name: surefire-reports
          path: target/surefire-reports/
```

The JDK is pinned to 11 because the IntroLab builds JHotDraw with Maven on JDK 11. `mvn verify` runs the full default lifecycle through compile, test, and package, plus any verification plugins bound to the verify phase.

The workflow YAML line by line

(beyond slides — practical knowledge.)

- `name:` — the label shown in the Actions tab; purely cosmetic but makes runs identifiable.
- `on: push / pull_request: branches: ["main"]` — the triggers. Every push to main and every pull request targeting main starts the pipeline. This is CILab step 3 ("build for each pull request", CILab p.1) and the principle "commit frequently, build every commit" (ContinuousIntegration p.3, p.5): nothing reaches the mainline without a verified build.
- `jobs: build: runs-on: ubuntu-latest` — the job executes on a fresh, disposable Linux virtual machine. A clean machine per run is what kills "the code builds on my box": the build server only gets code from the repository and is the final authority on whether the code builds (ContinuousIntegration p.6).
- `actions/checkout@v4` — clones the repository commit being verified into the runner. The `@v4` pins the action's major version so the pipeline does not silently change behaviour when the action updates.
- `actions/setup-java@v4` with `java-version: '11'`, `distribution: 'temurin'` — installs the pinned JDK; same compiler in CI as in the lab instructions, eliminating version-skew bugs.
- `cache: maven` — caches the local Maven repository (`~/.m2`) between runs keyed on `pom.xml`, so dependencies are not re-downloaded each time. This serves the fifth CI principle directly: keep the build fast (ContinuousIntegration p.3, p.11).
- `mvn --batch-mode --update-snapshots verify` — the heart. `--batch-mode` disables interactive prompts and progress spam (CI logs must be readable); `--update-snapshots` forces fresh snapshot dependencies; `verify` runs validate → compile → test → package → verify, so unit tests (Surefire) gate the build — the self-testing build principle (ContinuousIntegration p.9).
- `upload-artifact` with `if: always()` — publishes the Surefire test reports even when the build fails; a red build that hides its diagnostics defeats the feedback purpose of CI.

How does the pipeline work — the runtime walkthrough

Paste-ready answer:

When I push a commit or open a pull request, GitHub matches the event against the `on:` triggers in my workflow file and queues the job. GitHub Actions provisions a fresh `ubuntu-latest` virtual machine, so every run starts from a known-clean state with no leftovers from previous builds. The runner checks out exactly the commit under test, installs JDK 11, and restores the cached Maven dependencies. Then Maven executes the

lifecycle: it validates the POM, compiles the production sources, compiles the test sources, runs the whole JUnit test suite through the Surefire plugin, and packages the jar. If any step fails — compilation error, failing test, missing dependency — the run stops and the commit is marked with a red cross; the pull request shows the failure inline, and merging can be blocked until it is green. If everything passes, the run is marked green and the test reports are uploaded as an artifact. The whole loop — Commit → Build → Test → Report → back to Development — is the CI cycle from the deck, turning around the source repository as the single source of record (ContinuousIntegration p.2). The pipeline thus answers, automatically and within minutes of every change, the one question that matters in maintenance: does the system still work after my change?

Why did you structure it that way — stage-by-stage rationale

Paste-ready answer, one justification per stage:

- **Trigger on every pull request and push to main** — because integration problems grow with the time between integrations; building every commit shrinks the gap between defect introduction and detection, following "if it hurts, do it more often" (ContinuousIntegration p.7). I rejected a nightly build: a nightly batch can bury a day of commits in one failure, making the guilty change hard to identify.
- **Fresh virtual machine per run** — because the build server must be the neutral arbiter of "does it build"; a stateful build machine accumulates configuration that masks missing dependencies (ContinuousIntegration p.6). I rejected building on my own laptop: that is precisely the "works on my box" problem CI exists to kill.
- **Checkout then toolchain setup as separate, declarative steps** — so the pipeline definition documents its own environment; anyone can read the YAML and reproduce the build locally with the same JDK and Maven invocation.
- **Unit tests inside the build, not after it** — because a build that does not test proves only compilability; unit tests are fast and focused, which makes them the best method for verifying builds (ContinuousIntegration p.9). I rejected running only the slow end-to-end suite per commit: it would violate "keep the build fast" and developers would stop waiting for results.
- **Dependency caching** — purely in service of build speed, the fifth principle; the analysis-heavy parts of CI demand CPU and the deck's recurring imperative is "Please, KEEP IT FAST" (ContinuousIntegration p.11).
- **Test reports uploaded even on failure** — because the pipeline's product is feedback; a failure without diagnostics forces a local reproduction, doubling the feedback time.

The five CI principles — and how my pipeline realises each

The deck names five principles of Continuous Integration (ContinuousIntegration p.3). Paste-ready mapping:

1. **Environments based on stability** — code is promoted through progressively stricter environments (Dev → Test → Stage → Prod) as quality is proven (ContinuousIntegration p.4). In my setup, the pull-request build is the first gate; only verified code reaches the protected main branch, which represents the stable environment.
2. **Maintain a code repository** — one shared source of record; my workflow file itself lives in the repository, and the runner builds only from the repository (ContinuousIntegration p.3, p.6).
3. **Commit frequently and build every commit** — I committed small, functional changes, and the `on: push/pull_request` triggers build each one; small integrations keep each failure's blast radius tiny (ContinuousIntegration p.5).
4. **Make the build self-testing** — `mvn verify` runs the JUnit suite inside the build; this matters because individual programmers find fewer than half of their own bugs, and combining three or more quality

methods exceeds 90% defect removal (ContinuousIntegration p.8–9).

5. **Keep the build fast** — dependency caching, unit-test-first verification, and a lean job keep feedback within minutes; a slow build silently repeals principle 3 because developers stop waiting for it (ContinuousIntegration p.3, p.11).

Static analysis, coverage, and quality gates in the pipeline

The deck extends CI beyond pass/fail tests to automated quality (ContinuousIntegration p.10): static code analysis reads the source without running it to flag common Java bugs (FindBugs, PMD) and style violations (Checkstyle), while unit-test analysis measures coverage (Cobertura) and finds hotspots of low testing plus high complexity (SONAR). These run on each build so quality erosion is reported continuously rather than discovered at release. The deck's tool lists: test frameworks JUnit, NUnit, MSTest, Selenium, FitNesse (p.12); static analysis Checkstyle, CodeScanner, DRY, Crap4j, Findbugs, PMD, Fortify, Sonar, FXCop (p.13); coverage Emma, Cobertura, Clover, GCC/GCOV (p.14).

(beyond slides — practical knowledge) In a modern Maven workflow the same ideas are wired as plugins: SpotBugs (FindBugs's successor), Checkstyle, and PMD bind to the `verify` phase, and JaCoCo (Cobertura's successor) instruments the tests and can fail the build under a coverage threshold:

```
<plugin>
  <groupId>org.jacoco</groupId>
  <artifactId>jacoco-maven-plugin</artifactId>
  <executions>
    <execution><goals><goal>prepare-agent</goal></goals></execution>
    <execution><id>report</id><phase>verify</phase>
      <goals><goal>report</goal></goals></execution>
  </executions>
</plugin>
```

Justification sentence: "I added static analysis and coverage to the pipeline rather than relying on tests alone, because no single quality method is sufficient — combining three or more methods yields over 90% defect removal (ContinuousIntegration p.8) — and coverage makes the holes in my safety net visible before I refactor over them."

Jenkins and the alternatives — a comparison note

(beyond slides — practical knowledge, anchored in the deck's tool-neutral principles.)

GitHub Actions is one implementation of the CI principles; the principles themselves are tool-agnostic (ContinuousIntegration p.3). Jenkins is the classic self-hosted alternative: a build server you run yourself, configured with a `Jenkinsfile` pipeline-as-code that plays the same role as my workflow YAML (stages for checkout, build, test, publish). GitLab CI/CD uses `.gitlab-ci.yml`; Travis CI and CircleCI are hosted services with similar per-commit triggers. I chose GitHub Actions over Jenkins for three concrete reasons: first, the course repository already lives on GitHub, so the pipeline triggers natively on pull requests with zero webhook or server administration; second, a hosted runner gives me a fresh, reproducible build machine per run for free, whereas a self-hosted Jenkins accumulates state and must itself be maintained — extra maintenance burden in a course about reducing maintenance burden; third, the workflow file is versioned next to the code, so pipeline changes are reviewed in the same pull requests as code changes. Jenkins would be the better choice for builds needing special hardware, private network access, or organisation-wide shared build infrastructure. The equivalent Jenkinsfile sketch:

```
pipeline {
  agent any
  stages {
    stage('Build')  { steps { sh 'mvn -B compile' } }
    stage('Test')   { steps { sh 'mvn -B test' } }
    stage('Package'){ steps { sh 'mvn -B package' } }
  }
}
```

GitHub Packages and .maven-settings.xml — the dependency registry step

CILab step 4: to use shared jars from GitHub Packages, create a `.maven-settings.xml` file in the project root, following GitHub's "Working with the Apache Maven registry" guide (CILab p.1). The file tells Maven where to resolve (and authenticate against) the team's shared artifact registry:

```
<settings>
  <servers>
    <server>
      <id>github</id>
      <username>USERNAME</username>
      <password>${env.GITHUB_TOKEN}</password>
    </server>
  </servers>
</settings>
```

(beyond slides — practical knowledge: in the workflow you pass it as `mvn -s .maven-settings.xml verify`, and the `GITHUB_TOKEN` is injected by Actions as a secret — credentials never live in the repository.) Why it matters: the principle that the build server only gets code from the repository extends to dependencies — they must come from a registry, not from a jar someone copied onto a machine (ContinuousIntegration p.6). With the registry wired in, a clean runner can produce the complete build from nothing but the repository contents, which is the operational definition of a reproducible build.

Ready-made pipeline justification sentences

Each sentence stands alone, in the decision → rejected alternative → benefit pattern:

1. "I used GitHub Actions instead of a self-hosted Jenkins server because the repository already lives on GitHub; the pipeline triggers natively on every pull request, there is no build server to administer, and the workflow definition is version-controlled and code-reviewed alongside the code it builds."
2. "I triggered the build on every pull request rather than on a nightly schedule because building every commit shrinks the time between introducing and detecting a defect (ContinuousIntegration p.7) — with one commit per build, a red build identifies the guilty change immediately."
3. "I made the build self-testing by running the JUnit suite in the Maven verify phase rather than treating testing as a separate manual step, because a build that only compiles proves nothing about behaviour, and unit tests are fast and focused — the best method for verifying builds (ContinuousIntegration p.9)."
4. "I used Maven rather than hand-written build scripts because its standard lifecycle (compile → test → package → verify) gives every developer and the CI runner the identical, declarative build — the same `mvn verify` works on any machine, which eliminates 'works on my box' disputes (ContinuousIntegration p.6)."
5. "I cached the Maven dependency repository between runs instead of downloading dependencies every build, because feedback speed is a CI principle in its own right — a slow build quietly destroys the habit of committing frequently (ContinuousIntegration p.3, p.11)."

6. "I pinned the JDK version (11) and the action versions in the workflow instead of using 'latest', because a pipeline that changes its own environment over time produces failures unrelated to the code, eroding the team's trust in red builds."
7. "I kept the pipeline to build-plus-unit-tests on every commit and left slower end-to-end checks out of the per-commit path, because the deck's split is explicit: system tests are thorough but slow, unit tests are fast and focused and therefore verify builds (ContinuousIntegration p.9) — speed is what keeps the feedback loop alive."

Pipeline rapid Q&A — one-line answers

- **What is continuous integration?** The practice of merging all developers' working copies to a shared mainline several times a day, with each integration verified by an automated build (CILab p.1; ContinuousIntegration p.2).
- **Who coined the term?** Grady Booch, in his 1991 method — though he did not advocate integrating several times a day; Extreme Programming adopted CI and pushed the cadence to perhaps tens of times per day (CILab p.1).
- **What problem does CI solve?** Integration hell — the defect-laden big-bang merge after weeks of isolated work; constant integration keeps conflicts small and caught within hours (ContinuousIntegration p.2).
- **What is the CI cycle?** Commit → Build → Test → Report → back to Development, turning around source control as the single source of record (ContinuousIntegration p.2).
- **Name the five principles.** Environments based on stability; maintain a code repository; commit frequently and build every commit; make the build self-testing; keep the build fast (ContinuousIntegration p.3).
- **Why build every commit?** "If it hurts, do it more often" — frequent builds shrink the time between defect introduction and removal, so the cause is fresh when the failure appears (ContinuousIntegration p.7).
- **Why unit tests rather than system tests in the build?** Unit tests are fast (no database or file system) and focused (they pinpoint the broken unit) — the best method for verifying builds (ContinuousIntegration p.9).
- **Why is testing in the build essential?** Programmers are less than 50% efficient at finding their own bugs; combining three or more quality methods yields over 90% defect removal (ContinuousIntegration p.8).
- **What is a pipeline?** The ordered, automated sequence of stages (checkout → build → unit tests → static analysis → package) a change passes through on its way from commit toward release; each stage is a quality gate that stops a failing change. (beyond slides — practical phrasing of the deck's promotion-and-build concepts.)
- **What does "the build is the authority" mean?** The build server only gets code from the repository and is the final arbiter of stability — it settles "works on my machine" disputes neutrally (ContinuousIntegration p.6).
- **Where does the pipeline live in my repo?** In `.github/workflows/maven.yml`, version-controlled with the code; dependency registry credentials are wired through `.maven-settings.xml` in the project root (CILab p.1).

Technical decision justification bank

Every sentence below follows the mandated exam pattern — decision → rejected alternative → concrete benefits — and is usable verbatim with the [placeholders] filled in. The pattern comes straight from the lecturer's example: not "I used a switch statement" but "I used a switch statement instead of multiple if-statements because it improved readability and provided clearer structure for handling multiple conditions."

Justifications — naming

1. "I renamed [d] to [elapsedTimeInDays] instead of keeping the short name and adding a comment, because an intention-revealing name answers why it exists and what it does at every place it is used, while a comment explains it only where it was written (CleanCode p.11)."
2. "I named the new class [PenaltyCalculator] after its single responsibility rather than a vague name like [Manager] or [Processor], because a class name that states one responsibility makes the design legible from the package listing alone and exposes scope creep the moment someone tries to add unrelated methods (CleanCode p.21; SRP)."
3. "I used searchable constant names like [MAX_CLASSES_PER_STUDENT] instead of the magic number [7], because a named constant can be grepped and changed in one place, while a bare literal is invisible to search and its meaning dies with the author (CleanCode p.14; BetterCode p.58 — leave no magic constants behind)."
4. "I chose one word per concept — [fetch] everywhere, never a mix of [fetch]/[retrieve]/[get] for the same idea — instead of letting synonyms accumulate, because consistent vocabulary lets a reader transfer understanding from one class to the next without checking signatures (CleanCode p.18)."
5. "I renamed the method to a verb phrase [postPayment] and the boolean to a predicate [isPosted] rather than noun-ish ambiguity, because methods are actions and booleans are questions — names that read grammatically make call sites read like sentences (CleanCode p.21–22)."

Justifications — method extraction and size

6. "I extracted the [validation] block from [processOrder] into its own named method instead of leaving an explanatory comment above it, because the method name documents intent at every call site, cannot drift out of date like a comment, and brought the unit under the 15-line G1 threshold (BetterCode p.10; Refactoring1 p.11)."
7. "I split [method] into [stepA], [stepB] and [stepC] rather than keeping one long method, because short units are demonstrably easier to test, analyze, and reuse (BetterCode p.8) — each extracted step now has its own unit test, which the original monolith could not support."
8. "I kept each function at one level of abstraction, following the Stepdown Rule, instead of mixing business policy with string formatting in the same body, because mixed abstraction levels force the reader to mentally switch zoom levels mid-method (CleanCode p.24)."
9. "I replaced the boolean flag argument in [render(boolean isSuite)] with two named methods instead of keeping one method with a mode switch, because a flag argument loudly declares the function does more than one thing, and the two call sites now state their intent in their own names (CleanCode p.27)."
10. "I used guard clauses with early returns for the special cases in [method] instead of one nested if-pyramid with a single exit, because the un-indented mainline now shows the normal path at a glance and cyclomatic complexity fell below the G2 limit of 4 branch points (Refactoring1 p.34; BetterCode p.13)."

Justifications — class structure

11. "I applied Extract Class to split [GodClass] into [ClassA] and [ClassB] rather than reorganising methods within the one class, because the two responsibilities changed for different reasons at different times — after the split, each class has one reason to change (SRP) and the next change request touched exactly one of them (Refactoring1 p.17)."
12. "I moved [method] into [ClassB] (Move Method) instead of passing [ClassB]'s data out to it, because behaviour belongs next to the data it uses — the move eliminated four getter calls across the class boundary and the Feature Envy smell with them (Refactoring1 p.15)."
13. "I introduced a [ParameterObject] for the [start, end, min, max] cluster rather than keeping a five-parameter signature, because the values always travel together — the object meets G4's at-most-4-parameters rule and gave the missing concept a name and a home for its validation (BetterCode p.25; Refactoring1 p.41)."
14. "I replaced the inheritance of [Sub] from [Super] with delegation instead of overriding the unwanted methods to throw exceptions, because a subclass that refuses its parent's contract breaks Liskov substitution — with composition, [Sub] exposes only what it genuinely supports (Refactoring1 p.48; DesignPrinciplesAndPatterns p.8)."
15. "I made the implementation classes package-private behind a factory rather than leaving them public, because a smaller public surface is a smaller contract — clients now depend on the abstraction, and I can change implementations without a ripple (HighLevelRefactoring p.12, Indecent Exposure)."

Justifications — design patterns

16. "I used the Strategy pattern for the [pricing] variants instead of a switch statement over a type code, because each algorithm became a small, separately testable class and adding a variant is now a new class plus one registration — the switch would have to be found and edited in every place it was duplicated (HighLevelRefactoring p.12; Refactoring1 p.35)."
17. "I used the Observer pattern to let [View] track [Model] instead of having [Model] call [View] directly, because the subject must not know its observers' concrete types — the model now compiles without the GUI and can be tested headlessly (DesignPrinciplesAndPatterns p.30; JHotDraw's figure-listener design works the same way)."
18. "I used a Factory Method for creating [Product] objects instead of scattering `new [ConcreteProduct]()` across client code, because centralised creation means a new product type changes one factory, not every client — creation knowledge stops sprawling (HighLevelRefactoring p.17–19; Ker05, Move Creation Knowledge to Factory)."
19. "I used the Adapter pattern to fit [LegacyClass] to the [Target] interface instead of rewriting the legacy class or editing all clients, because adaptation isolates the mismatch in one class — both sides stay unchanged, which minimised the diff and the regression risk (DesignPrinciplesAndPatterns p.29)."
20. "I used the Command pattern for the editor actions instead of an if-else dispatcher, because each action object carries its own state and undo hook — which is exactly how JHotDraw structures its menu commands — and the dispatcher stopped growing with every new action (HighLevelRefactoring p.12)."
21. "I used the Template Method pattern for the shared [export] skeleton instead of copying the algorithm into each subclass, because the invariant step order now lives in one place and subclasses override only the steps that genuinely differ (Refactoring1 p.47)."

Justifications — error handling

22. "I used exceptions instead of returned error codes, because error codes force every caller into immediate if-checks that bury the happy path, while exceptions separate the normal flow from error processing and

cannot be silently ignored (CleanCode p.65)."

23. "I extracted the try/catch bodies into their own methods so that error handling is one thing the method does, rather than mixing transaction logic and recovery logic in one body — functions should do one thing, and handling errors is one thing (CleanCode p.66–67)."
24. "I returned an empty list (a special-case object) from [finder] instead of null, because every null return obligates every caller to a null check forever — don't return null, don't pass null — and one forgotten check is a NullPointerException in production (CleanCode p.70–72; Refactoring1 p.36)."
25. "I validated [input] at the system boundary and threw [IllegalArgumentException] with a descriptive message instead of letting bad data flow inward, because failing fast at the boundary localises the fault to its source, while a deep failure surfaces far from its cause and costs the next maintainer the whole diagnosis."

Justifications — tests

26. "I wrote the unit test for [unit] before fixing the bug rather than after, because a test that fails first proves it actually detects the defect — a test written after the fix may pass vacuously and guard nothing (CleanCode p.74, the Three Laws of TDD)."
27. "I used AssertJ's fluent assertions instead of bare JUnit asserts, because `assertThat(result).containsExactly(...)` reads as the specification it checks and its failure messages name what differed — a failing build then explains itself (BDD p.22–25)."
28. "I mocked the [DataServer] dependency with Mockito instead of testing against the real service, because the unit test must be fast and deterministic — no database, no network — and the mock lets me verify the interaction contract precisely (Software Testing deck p.57–66; ContinuousIntegration p.9)."
29. "I kept one logical assertion per test and named each test for the behaviour it checks, instead of one giant test method asserting everything, because when a focused test fails its name alone tells me which behaviour broke — F.I.R.S.T.: fast, independent, repeatable, self-validating, timely (CleanCode p.76)."

Justifications — commits and branches

30. "I committed in small, functional increments with descriptive messages instead of batching a week of work into one commit, because small commits keep each integration's blast radius tiny, make `git log` a readable history of the change process, and let me revert a single wrong step without losing the rest (ContinuousIntegration p.5; Git Lab p.11)."
31. "I developed each change on a feature branch and merged via pull request instead of committing straight to main, because the pull request is the gate where the CI build verifies the change before integration — main stays releasable at all times (IntroLab p.1, GitHub flow; ContinuousIntegration p.4)."
32. "I wrote commit messages that state the why, not just the what — '[Refactor: extract PenaltyCalculator from Library (G1 violation)]' instead of '[changes]' — because the repository is evaluated as part of the portfolio and the history should document the maintenance process itself (Git Lab p.5)."
33. "I kept generated artifacts and large binaries out of version control instead of committing build output, because the repository should hold the human-authored sources from which everything else is generated — committing an EXE or a 2 GB binary bloats every clone forever (Git Lab p.6)."

Justifications — pipeline

34. "I configured the pipeline to build on every pull request instead of building manually before releases, because building every commit shrinks the gap between defect introduction and detection to minutes (ContinuousIntegration p.7) — by the time a defect is found, its cause is still fresh in my head."

35. "I made the pipeline self-testing with `mvn verify` rather than a compile-only check, because a green compile proves syntax, not behaviour; unit tests in the build are what turn 'it builds' into 'it works' (ContinuousIntegration p.9)."
36. "I let the CI runner — not my laptop — be the authority on whether the project builds, because the build server only gets code from the repository and therefore settles every 'works on my machine' dispute neutrally (ContinuousIntegration p.6)."

Justifications — architecture boundaries

37. "I kept all dependencies pointing inward toward the domain — the Dependency Rule — instead of letting business rules import UI or database types, because source code dependencies that cross boundaries inward only mean the policy core can be compiled, tested, and reused without any frameworks attached (Clean Architecture deck, p.8)."
38. "I treated the database as a detail behind an [EntityGateway] interface instead of calling persistence directly from the use case, because the gateway inverts the dependency (DIP): the domain owns the interface, the database implements it, and swapping storage technologies no longer touches a single business rule (Clean Architecture deck; DesignPrinciplesAndPatterns p.12)."
39. "I depended on abstractions at every hinge point — interfaces owned by the client package — rather than on concrete classes from other components, because stable abstractions absorb change: the volatile implementation can churn behind the interface while every dependent module stays untouched (DesignPrinciplesAndPatterns p.12–14, p.28)."
40. "I split the GUI from the domain logic (Separate Domain from Presentation) instead of leaving rules inside event handlers, because logic inside widgets can only be tested through the GUI — after the split, the same rules are plain JUnit tests and the editor window is a thin shell (Refactoring1 p.56)."

Testing snippet bank

JUnit 5 unit test anatomy — annotated example

(beyond slides — the deck demonstrates JUnit 4 lifecycle annotations; this is the JUnit 5 equivalent used in current Maven setups, with the JUnit 4 names noted.)

```
import org.junit.jupiter.api.*;
import static org.assertj.core.api.Assertions.assertThat;

class FixedSizeQueueTest {

    private FixedSizeQueue queue;          // fresh fixture per test

    @BeforeEach                            // JUnit 4: @Before
    void setUp() {
        queue = new FixedSizeQueue(2);     // Arrange (shared)
    }

    @Test
    @DisplayName("enqueue adds the element at the tail")
    void enqueue_addsElementAtTail() {
        queue.enqueue(5);                  // Act
        assertThat(queue.size()).isEqualTo(1);   // Assert
    }

    @Test
    void enqueue_onFullQueue_throws() {
        queue.enqueue(1);
```

```
        queue.enqueue(2);
        Assertions.assertThrows(IllegalStateException.class,
                () -> queue.enqueue(3));   // exception-testing idiom
    }

    @AfterEach                             // JUnit 4: @After
    void tearDown() { queue = null; }
}
```

Anatomy: the class groups tests for one unit under test (the queue — the deck's running SUT, Software Testing deck p.11); `@BeforeEach` rebuilds the fixture so tests stay independent (the I in F.I.R.S.T., CleanCode p.76); each `@Test` is one behaviour with Arrange–Act–Assert structure; `assertThrows` replaces JUnit 4's `@Test(expected=...)`. Surefire runs all of this inside `mvn test`, which is what makes the CI build self-testing (ContinuousIntegration p.9).

AssertJ fluent assertions — the greatest hits

AssertJ is the course's assertion library — fluent, readable assertions that make the Then step read like the specification (BDD p.21–25):

```
import static org.assertj.core.api.Assertions.*;

assertThat(frodo.getName()).isEqualTo("Frodo");          // equality
assertThat(frodo).isNotEqualTo(sauron);                  // object identity
assertThat(frodo.getName()).startsWith("Fro")            // chaining: one subject,
                           .endsWith("do")               // several conditions,
                           .isEqualToIgnoringCase("frodo"); // one readable sentence

assertThat(fellowship).hasSize(9)                        // collections
                      .contains(frodo, sam)
                      .doesNotContain(sauron);

assertThat(list).extracting(Character::getName)          // extract a property
                .containsExactly("Frodo", "Sam", "Pippin");

assertThatThrownBy(() -> queue.dequeue())                // exceptions, fluently
        .isInstanceOf(IllegalStateException.class)
        .hasMessageContaining("empty");
```

Why AssertJ over plain JUnit asserts: the assertion reads left-to-right as a sentence (`assertThat(subject).expectation()`), the IDE autocompletes the legal expectations for the subject's type, and failure messages name exactly what differed — `assertEquals(a, b)` argument-order mistakes disappear (BDD p.22–23). Custom Conditions and custom assertions extend the vocabulary with domain words (BDD p.26–28).

Test doubles — stub, mock, spy with Mockito

The deck distinguishes the doubles by purpose (Software Testing deck p.57–75): a **stub** feeds canned answers to the unit under test; a **mock** additionally records interactions so you can verify them; a **spy** wraps a real object, letting most calls through while you observe or override some.

```
import static org.mockito.Mockito.*;

// STUB usage: canned answers
DataServer server = mock(DataServer.class);
when(server.getData()).thenReturn(List.of("a", "b"));
```

```
Library library = new Library(server);     // inject the double
library.refresh();

// MOCK usage: verify the interaction contract
verify(server).getData();                  // was it called?
verify(server, times(1)).getData();        // exactly once?
verifyNoMoreInteractions(server);          // and nothing else

// SPY: real object, observed
List<String> spyList = spy(new ArrayList<>());
spyList.add("x");
verify(spyList).add("x");                  // real behaviour ran AND was recorded
```

Why doubles at all: the unit test must be fast, focused, and deterministic — no database, no network, no shared state (ContinuousIntegration p.9). The double replaces the slow or unpredictable collaborator so a failure pinpoints the unit, not the environment. Design-for-testability makes this possible: dependencies arrive through constructors or setters (so they can be substituted), not through `new` calls buried in the body (Software Testing deck p.14).

Parameterized tests — one test, many cases

(beyond slides — practical knowledge; the deck motivates the idea via equivalence partitioning and boundary-value analysis, p.10–13.)

```
import org.junit.jupiter.params.ParameterizedTest;
import org.junit.jupiter.params.provider.CsvSource;

class FineCalculatorTest {

    @ParameterizedTest(name = "{0} days late costs {1}")
    @CsvSource({
        "0,  0.00",     // boundary: not late at all
        "1,  0.50",     // boundary: first late day
        "14, 7.00",     // partition: ordinary lateness
        "15, 10.00",    // boundary: surcharge kicks in
        "16, 10.50"     // partition: past the surcharge
    })
    void fine_isComputedFromDaysLate(int daysLate, double expected) {
        assertThat(new FineCalculator().fineFor(daysLate))
                .isEqualTo(expected);
    }
}
```

The case table operationalises the deck's two black-box design techniques: each row is either a representative of an equivalence partition (inputs the spec treats identically — test one per partition) or a boundary value (where off-by-one faults cluster — test each edge and its neighbours) (Software Testing deck p.10–13). The justification sentence: "I chose test inputs by equivalence partitioning and boundary-value analysis instead of ad-hoc values, because testing cannot be exhaustive (the deck's Turing point) — partitions maximise the distinct behaviours covered per test, and boundaries are where the faults live."

JGiven scenario — the full given/when/then class

JGiven is the course's developer-friendly BDD framework: scenarios are pure Java, so they refactor and compile with the production code, unlike plain-text frameworks whose text and step definitions drift apart (BDD p.6–9). A complete scenario test plus its three stage classes, following the deck's canonical example (BDD p.10, p.14–16):

```
public class CookingScenarioTest
        extends ScenarioTest<GivenIngredients, WhenCook, ThenMeal> {

    @Test
    public void a_pancake_can_be_fried_out_of_an_egg_milk_and_flour() {
        given().an_egg()
               .and().some_milk()
               .and().the_ingredient("flour");
        when().the_cook_mangles_everything_to_a_dough()
              .and().the_cook_fries_the_dough_in_a_pan();
        then().the_resulting_meal_is_a_pan_cake();
    }
}

public class GivenIngredients extends Stage<GivenIngredients> {
    @ProvidedScenarioState
    List<String> ingredients = new ArrayList<String>();

    public GivenIngredients an_egg()      { return the_ingredient("egg"); }
    public GivenIngredients some_milk()   { return the_ingredient("milk"); }
    public GivenIngredients the_ingredient(String ingredient) {
        ingredients.add(ingredient);
        return this;                      // fluent chaining
    }
}

public class WhenCook extends Stage<WhenCook> {
    @ExpectedScenarioState List<String> ingredients;  // from Given
    @ProvidedScenarioState Set<String> dough;
    @ProvidedScenarioState String meal;
    @ScenarioState Cook cook;

    public WhenCook the_cook_mangles_everything_to_a_dough() {
        dough = cook.makeADough(ingredients);  return this;
    }
    public WhenCook the_cook_fries_the_dough_in_a_pan() {
        meal = cook.fryDoughInAPan(dough);     return this;
    }
}

public class ThenMeal extends Stage<ThenMeal> {
    @ExpectedScenarioState String meal;       // from When

    public void the_resulting_meal_is_a_pan_cake() {
        assertThat(meal).isEqualTo("pancake"); // AssertJ does the checking
    }
}
```

State flows between stages purely through the annotations: Given *provides* the ingredients, When *expects* them and *provides* the meal, Then *expects* the meal and asserts with AssertJ (BDD p.12–16). JGiven generates console and HTML5 reports from the method names — living documentation a domain expert can read (BDD p.17–18).

Test naming and the AAA pattern

(beyond slides — practical knowledge, consistent with the deck's clean-test material.)

Every test follows Arrange–Act–Assert: build the fixture, perform exactly one action, assert the observable outcome. The name states behaviour, not implementation — a reader should learn the unit's contract from the test list alone:

```
@Test void borrow_setsBookUnavailable() { /* ... */ }
@Test void borrow_whenBookAlreadyBorrowed_throws() { /* ... */ }
@Test void returnBook_beforeDueDate_incursNoFine() { /* ... */ }
```

The naming scheme `methodUnderTest_condition_expectedOutcome` makes a red test self-explaining in the CI report: `borrow_whenBookAlreadyBorrowed_throws FAILED` is a bug report in one line. Pair it with one logical assertion per test (one concept per test, CleanCode p.76): when a test can fail for only one reason, the failure needs no debugging to interpret. In BDD terms, Arrange–Act–Assert and Given–When–Then are the same skeleton in different vocabularies (BDD p.5) — I used GWT wording for behaviour-level scenarios and AAA for unit-level tests.

First-person testing reflections

1. "In my testing lab I wrote JUnit tests for [Class] before touching its code, because the test suite is what makes refactoring's behaviour-preservation verifiable rather than hoped-for (BetterCode p.54, G9) — every Extract Method I performed afterwards was followed by a green re-run."
2. "I designed the test inputs for [unit] with equivalence partitioning and boundary-value analysis: one representative per partition of [input domain] plus the boundaries [edges], because faults cluster at boundaries and testing cannot be exhaustive (Software Testing deck p.10–13)."
3. "When my test for [behaviour] failed, I followed the disciplined diagnosis before changing code: is the fault in the SUT, in the test itself, or in the specification my oracle encodes? In my case it was [which], which I confirmed by [how] — fixing the right artifact instead of patching the symptom."
4. "I mocked [collaborator] with Mockito in the [Class] tests instead of instantiating the real one, because the real [collaborator] needs [database/network/GUI]; with the mock, the suite runs in [seconds] and a failure can only mean [Class] itself broke — which is exactly what a unit test should mean."
5. "I wrote a JGiven scenario for the [feature] user story so the acceptance criterion from my change request became an executable specification: the given/when/then method names mirror the story card, and the generated HTML report documents the behaviour for a non-programmer reader (BDD p.4, p.17–18)."
6. "After every refactoring commit, the CI pipeline re-ran my whole suite as a regression test, because the maintenance risk is not the change you made but the behaviour you broke elsewhere — automated regression is what lets me change [Class] without re-verifying the system by hand (ContinuousIntegration p.9)."

Why testing matters — justification sentences

1. "Testing was important in my lab because refactoring is defined as changing structure without changing observable behaviour — without an automated suite, 'behaviour preserved' is an assertion of faith; with one, it is a green build (Refactoring1 p.5; BetterCode p.54)."
2. "I automated the tests rather than re-testing manually because automation makes testing repeatable and cheap enough to run on every commit; manual testing is unrepeatable, unrecorded, and skipped exactly when time pressure makes it most necessary (BetterCode p.54; ContinuousIntegration p.9)."
3. "Tests document the code: each test shows how the unit is meant to be called and what it promises — when I returned to [Class] weeks later, its test class was the fastest accurate description of its contract available (BetterCode p.54)."
4. "Writing tests made me write better code, because testability forces decoupling — to test [Class] in isolation I had to inject its [dependency] through the constructor, and that same seam later let me swap [implementation] without touching the class (BetterCode p.54; Software Testing deck p.14)."
5. "A passing suite cannot prove correctness — exhaustive testing is impossible — but it proves the absence of every defect I knew to look for, and each bug found in use became a new test that can never regress silently

(Software Testing deck p.3–6)."

6. "Unit tests, not system tests, gate my CI build because they are fast and focused: no database or file system, and a failure pinpoints the broken unit — speed keeps the build-every-commit habit alive, focus makes red builds actionable (ContinuousIntegration p.9)."

TDD — red, green, refactor, with the Borrow Book shape

The Test-Driven Development cycle (Software Testing deck p.33–49): write a failing test (red), write the minimum production code to pass it (green), then refactor both code and tests while staying green. The Three Laws keep the loop honest: no production code except to pass a failing test; only enough test to fail; only enough production code to pass (CleanCode p.74). The deck's flagship example is Borrow Book: the first test asserts a freshly borrowed book is unavailable, drives the creation of `borrow()` and `isAvailable()`, and each subsequent test (borrowing an already-borrowed book, returning, overdue handling) forces the next slice of behaviour into existence.

```
@Test
void borrowedBook_isNoLongerAvailable() {     // RED: borrow() doesn't exist yet
    Book book = new Book("Refactoring");
    book.borrow();                            // GREEN: implement minimally
    assertThat(book.isAvailable()).isFalse(); // REFACTOR: clean up, stay green
}
```

Why it matters for maintenance: TDD yields a regression suite as a by-product of development, keeps design testable by construction (you feel untestable coupling immediately, while it is cheap to fix), and the refactoring step is built into the rhythm rather than deferred. Paste-ready justification: "I worked test-first on [feature] instead of testing after, because the failing test proves the test can detect the defect, the minimal implementation prevents speculative code, and the refactor step let me clean [Class] with the safety net already in place (Software Testing deck p.33–40)."

Testing a GUI — under the GUI, and AssertJ-Swing

The deck's guidance on GUI testing (Software Testing deck p.24–32): test *under* the GUI where possible — drive the domain logic through its API rather than through pixel coordinates — because record-and-play GUI scripts are the canonical fragile tests, breaking on every cosmetic change. This is why Separate Domain from Presentation (Refactoring1 p.56) is also a testing strategy: once the rules leave the widgets, they are plain unit-testable.

For behaviour that genuinely lives in the GUI, the course names AssertJ-Swing (BDD p.29) — relevant to a JHotDraw-style editor:

```
FrameFixture window = new FrameFixture(robot, editorFrame);
window.menuItemWithPath("File", "New").click();
window.panel("drawingPanel").click();
window.label("statusBar").requireText("1 figure");
```

The robot drives real Swing events and the fixtures assert on component state, so the test exercises the actual wiring — at the price of speed and brittleness, which is why only the thin GUI layer is tested this way while everything else runs headless. Paste-ready justification: "I tested [rules] under the GUI as plain JUnit tests and reserved AssertJ-Swing for the [interaction] that only exists in the view, because GUI-driven tests are slow and fragile — minimising them keeps the suite fast while the wiring still gets covered (Software Testing deck p.24–32; BDD p.29)."

Clean Code & Architecture applied

The Boy Scout Rule — applied

Concept. "Leave the campground cleaner than you found it" — every time you touch code, leave it slightly better than you found it (CleanCode p.3). The SIG operationalises the same ethic as G10: don't leave code smells behind after development work, itemised as seven leave-nothing-behind rules — no unit-level smells, no bad comments, no commented-out code, no dead code, no long identifiers, no magic constants, no badly handled exceptions (BetterCode p.57–58). This is postfactoring as a personal discipline. **Applied to my lab.** "Whenever a change took me into a JHotDraw class, I budgeted a small Boy-Scout pass: in [Class] I [renamed a cryptic local / deleted commented-out code / replaced a magic constant] even though the change request did not demand it — small messes compound into unmaintainable code precisely because each one alone seems too small to fix."

Meaningful names — applied

Concept. Names should reveal intention — why it exists, what it does, how it is used; avoid disinformation; make meaningful distinctions; use pronounceable, searchable names; avoid encodings and mental mapping; class names are nouns, method names are verbs; one word per concept (CleanCode p.11–22). The deck's flagship example: `int d; // elapsed time in days` versus `int elapsedTimeInDays` — the comment becomes unnecessary the moment the name does its job. **Applied to my lab.** "In my refactoring lab I renamed [old name] to [new name] in [Class] because the original required a mental map from abbreviation to meaning; after the rename, every use site documents itself, and a grep for the concept actually finds it."

Small functions that do one thing — applied

Concept. The first rule of functions: they should be small; the second: smaller than that. A function should do one thing, do it well, and do it only — if you can extract another function from it with a name that is not merely a restatement, it was doing more than one thing (CleanCode p.23–24). The SIG version is measurable: at most 15 lines of code per unit (G1, BetterCode p.10). **Applied to my lab.** "[Method] in [Class] did [N] things; I extracted [stepA] and [stepB] until each function did one thing stated by its name. The proof of 'one thing' was the tests: each extracted function got a focused unit test that the original tangle could not support."

One level of abstraction and the Stepdown Rule — applied

Concept. Statements within a function should all sit at the same level of abstraction, and code should read top-down like a narrative: each function followed by those at the next level down, so the program reads as a set of TO-paragraphs (CleanCode p.24). **Applied to my lab.** "After my extraction pass, [Class] reads top-down: [highLevelMethod] tells the story in domain words, and each helper beneath it descends one level. A reader can stop at any depth and still hold a true picture — which is exactly what partial comprehension in maintenance requires."

Function arguments — applied

Concept. The ideal number of arguments is zero, then one, then two; three should be avoided where possible. Flag arguments are ugly — passing a boolean loudly proclaims the function does more than one thing. Argument objects: when a function needs more than two or three values, some of them likely belong in a concept of their own (CleanCode p.25–28). SIG G4 sets the hard ceiling at 4 (BetterCode p.25). **Applied to my lab.** "I replaced the flag argument in [method(boolean)] with two intention-named methods, and bundled the [x, y, w, h] quartet into a [Bounds] object — every signature in the class now meets G4 and reads as prose at the call site."

Command-Query Separation — applied

Concept. Functions should either do something or answer something — change the state of an object, or return information about it, but not both. A method that does both invites bizarre call sites like `if (set("username", "bob"))` (CleanCode p.30). **Applied to my lab.** "[Method] both returned [value] and mutated [state]; I split it into a pure query and a command (Separate Query from Modifier, Refactoring1 p.38), after which the query became safe to call from assertions and logging without side effects."

Comments — the good, the bad, applied

Concept. "Don't comment bad code — rewrite it." The proper use of comments is to compensate for our failure to express ourselves in code — every comment is a failure we tolerate only when code cannot speak (CleanCode p.34–35). Good comments: legal headers, informative regex explanations, explanation of intent, warnings of consequences, amplification, javadoc on public APIs (p.35–37). Bad comments: mumbling, redundant, misleading, mandated, journal, noise, position markers, attributions, commented-out code, HTML in comments, nonlocal information (p.38–52). **Applied to my lab.** "I deleted the commented-out block in [Class] — version control already remembers it — and replaced the what-comment over the [logic] block with an extracted method whose name says the same thing; the one comment I kept explains why [decision], which no code can express."

Formatting and the newspaper metaphor — applied

Concept. Code should read like a newspaper article: name as headline, high-level synopsis at top, detail increasing downward. Vertical openness separates concepts; vertical density groups related lines; variables declared close to use; instance variables at the top; caller above callee. A team's style rules beat any individual's preferences (CleanCode p.53–60). **Applied to my lab.** "I reordered [Class] so public behaviour sits above private helpers in call order, and let the IDE enforce the shared format — formatting is about communication, and consistent layout is the cheapest readability gain in the whole catalog."

Error handling — applied

Concept. Prefer exceptions to error codes; extract try/catch into functions of their own (error handling is one thing); define the normal flow with special-case objects; don't return null; don't pass null (CleanCode p.65–72). Badly handled exceptions are one of G10's seven sins (BetterCode p.58). **Applied to my lab.** "[Method] returned [-1/null] as a failure signal that two callers ignored; I converted it to throw [Exception] with a message naming the offending [input], and gave [finder] an empty-collection return so its callers lost their null checks. Failures now announce themselves at the fault, not three frames later."

Objects, data structures, and the Law of Demeter — applied

Concept. Objects hide their data and expose behaviour; data structures expose data and have no behaviour — the anti-symmetry. The Law of Demeter: a method should talk only to friends — its own class, its parameters, objects it creates, its fields — not to strangers reached through friends. Train wrecks like `a.getB().getC().doIt()` couple the caller to the whole object graph (CleanCode p.61–64). **Applied to my lab.** "[Client] navigated `[a.getB().getC()]` to reach [behaviour]; I applied Hide Delegate so [a] answers directly. The client now survives any reshaping of the intermediate structure — the chain was an undeclared dependency on three classes."

Unit tests in Clean Code — TDD laws and F.I.R.S.T. — applied

Concept. The Three Laws of TDD: write no production code except to pass a failing test; write only enough test to fail; write only enough production code to pass (CleanCode p.74). Test code is as important as production code — dirty tests rot first and get deleted. One assert/one concept per test. F.I.R.S.T.: Fast, Independent, Repeatable, Self-validating, Timely (CleanCode p.75–77). **Applied to my lab.** "I kept my JHotDraw tests to one concept each and independent of execution order; when [test] needed [another test]'s leftovers I knew the fixture was wrong, not the order. The suite runs in [seconds], which is why it actually gets run."

Classes — small, SRP, cohesion, and emergent design — applied

Concept. Classes should be small, measured in responsibilities; the Single Responsibility Principle says one reason to change; high cohesion means methods and variables co-depend as a logical whole. The four rules of simple design, in priority order: runs all the tests; contains no duplication; expresses the intent of the programmer; minimises the number of classes and methods (CleanCode p.78–81). **Applied to my lab.** "After Extract Class, [ClassA] and [ClassB] each have one describable responsibility and every field is used by most methods — cohesion went from incidental to definitional. I checked the result against the four rules of simple design in order: green tests first, then no duplication, then expressive names, and only then worried about class count."

The four symptoms of rotting design — applied

Concept. Rigidity — the software is hard to change because every change forces other changes; Fragility — changes break things in conceptually unrelated places; Immobility — components cannot be reused elsewhere because of entanglement; Viscosity — doing things right is harder than doing things wrong, in design and environment alike (DesignPrinciplesAndPatterns p.1–4). Rot is driven by changing requirements interacting with dependency structure — the cure is dependency management, not blaming the requirements. **Applied to my lab.** "Before refactoring, [area] showed rigidity — my [small change] cascaded into [N] classes — and viscosity, since the hack [doing X] was easier than the right change. After [refactoring], the same category of change is additive: a new [variant] is one class, no cascade."

Single Responsibility Principle (SRP) — applied

Concept. A class should have one, and only one, reason to change. Responsibilities are axes of change; coupling two responsibilities into one class couples their change schedules (DesignPrinciplesAndPatterns; OOPrinciples deck). The micro form of the same idea is Divergent Change; the metric echo is G5's separation of concerns in modules (BetterCode p.31). **Applied to my lab.** "[Class] changed both when [UI concern] changed and when [domain rule] changed — two masters. I split it so each new class answers to one; the immediate payoff was that [change request] produced a one-class diff and a one-class test update."

Open/Closed Principle (OCP) — applied

Concept. Software entities should be open for extension but closed for modification — you add behaviour by adding new code, not by editing working code. The mechanism is abstraction: clients depend on an abstract interface, and new variants implement it (DesignPrinciplesAndPatterns p.5–7, the modem example). **Applied to my lab.** "The type-switch on [typeCode] meant every new [kind] edited tested code in [N] places. After Replace Conditional with Polymorphism, the dispatch is closed and the hierarchy is open: my [new kind] commit added [NewClass] and touched nothing else — the diff is the principle made visible."

Liskov Substitution Principle (LSP) — applied

Concept. Subtypes must be substitutable for their base types: any code that works with the base must work, unsurprised, with every derivative. The classic violation is Circle/Ellipse — mathematically an is-a, behaviourally not, because setWidth/setHeight promises break (DesignPrinciplesAndPatterns p.8–12). Validity is defined by the clients' reasonable assumptions, not by the hierarchy diagram. **Applied to my lab.** "[Sub] overrode [method] to throw, surprising every caller of [Super] — a textbook refused bequest. I replaced the inheritance with delegation so [Sub] only advertises what it honours; no client of [Super] can now receive an object that breaks the contract."

Interface Segregation Principle (ISP) — applied

Concept. Clients should not be forced to depend on methods they do not use. Fat interfaces couple their clients to each other: when one client's needs change the interface, all other clients must recompile and redeploy. Split fat interfaces into client-specific ones (DesignPrinciplesAndPatterns p.15–16). **Applied to my lab.** "[Client] consumed only [n] of [Interface]'s [m] methods; I extracted [RoleInterface] for that slice (Extract Interface, Refactoring1 p.46). Besides honesty, the narrow interface made the unit test trivial — the mock implements three methods, not seventeen."

Dependency Inversion Principle (DIP) — applied

Concept. High-level modules should not depend on low-level modules; both should depend on abstractions. Abstractions should not depend on details; details should depend on abstractions. The "inversion" is of the traditional layering where policy imports implementation (DesignPrinciplesAndPatterns p.12–14). **Applied to my lab.** "[UseCase] called [ConcreteStore] directly, so the policy depended on the detail. I introduced [StoreInterface] owned by the use-case package and made [ConcreteStore] implement it — the source-code arrow now points from detail to policy, and the use-case tests run against an in-memory fake."

Clean Architecture — layers and the Dependency Rule — applied

Concept. Clean Architecture arranges the system in concentric circles — Entities, Use Cases, Interface Adapters, Frameworks and Drivers — governed by one rule: source-code dependencies point only inward, toward higher-level policy. Nothing in an inner circle knows anything about an outer circle. Inputs and outputs cross boundaries as simple request/response models through interfaces (Boundaries), and the Interactor orchestrates the use case (Clean Architecture deck p.8 and figures; L05 Concepts 8–9). **Applied to my lab.** "I checked my change against the Dependency Rule: [domain class] imports no Swing, no persistence, no framework type. Where the use case needed [outer service], I gave the inner circle an interface and the outer circle the implementation — dependency inversion at the boundary, exactly as the diagram from Itslearning prescribes."

The database and the UI are details — applied

Concept. In Clean Architecture the database is a detail — a plug-in to the business rules, reached through an Entity Gateway interface; the UI equally so. The successful architecture's four characteristics: independent of frameworks, testable without UI or database, independent of UI, independent of database (Clean Architecture deck; L05 Concepts 7, 10). **Applied to my lab.** "I kept [business rule] testable with neither GUI nor storage attached: its tests construct the entities directly and stub the gateway. When the lab asked me to [swap/extend persistence or UI], the business-rule package compiled untouched — the practical cash value of treating IO as a plug-in."

Package principles and stable abstractions — applied

Concept. Package cohesion: REP (the granule of reuse is the granule of release), CCP (classes that change together stay together), CRP (classes used together are reused together). Package coupling: ADP (no dependency cycles), SDP (depend in the direction of stability), SAP (a package should be as abstract as it is stable) — stable packages should be abstract so stability does not block extension; instability is where the concrete churn belongs (DesignPrinciplesAndPatterns p.17–27, with the Ca/Ce/I/A/D metric set). **Applied to my lab.** "My [core] package is depended on by [n] others, so I kept it abstract — interfaces and entities only — and pushed concrete, volatile classes outward to the unstable edge (SAP/SDP). When I found the cycle [A → B → A] I broke it with an interface owned by the depender (ADP, dependency inversion), so a release of one no longer drags the other."

GRASP — assigning responsibilities — applied

Concept. GRASP names nine responsibility-assignment patterns (OOPrinciples deck; L05 Concept 19): Information Expert (give the responsibility to the class with the information), Creator (B creates A if B contains, aggregates, records, or closely uses A), Controller (a non-UI object receives system events), Low Coupling and High Cohesion (the two evaluative principles), Polymorphism (type-based alternatives via operations, not conditionals), Pure Fabrication (invent a class to preserve cohesion), Indirection (an intermediary decouples), Protected Variations (wrap predicted instability behind a stable interface). **Applied to my lab.** "When deciding where [newMethod] belonged, I applied Information Expert: [Class] already holds [data], so it computes [result] — the alternative, computing it in [OtherClass] from getters, would have been Feature Envy by another name. For creating [Thing] I followed Creator and let [Container] do it, since it aggregates them; and the [VariablePoint] I judged likely to change got Protected Variations: an interface in front, so future variation hits one seam."

MVC and Observer in JHotDraw — applied

Concept. JHotDraw is built on Model-View-Controller: figures are the model, drawing views render them, tools act as controllers translating input into model changes. The hinge is the Observer pattern (DesignPrinciplesAndPatterns p.30): views register as listeners on the drawing; figures announce changes through figure-changed events; no figure knows any view's concrete type. This is why Separate Domain from Presentation and Duplicate Observed Data (Refactoring1 p.27, p.56) feel native in this codebase — its architecture already separates the layers. **Applied to my lab.** "My change to [figure/feature] respected the MVC split: I modified the model class [Figure] and let the existing listener chain refresh every view, rather than calling the view directly — which would have reversed a dependency the architecture deliberately points one way. When I needed [the view to react specially], I subscribed it as another observer instead of teaching the model about it, keeping the model compilable and testable without any GUI on the classpath."

Reflection paragraph templates per lab

Git lab — version control and repository setup

Fill-in paragraph (120–180 words):

"In the Git lab I set up the version-control foundation for all later work. I forked the JHotDraw repository on GitHub, cloned my fork locally, and built it with Maven on JDK 11, verifying the build by running the [SVG sample] application (IntroLab p.1). I practised the core workflow across Git's four areas — workspace, index, local repository, remote — using add, commit, push, fetch, merge and pull (Git Lab p.11). For every subsequent change I followed GitHub flow: a feature branch [branch name] per change, merged to main

through a pull request. I committed in small, functional increments with messages explaining why, such as '[example commit message]', because the repository itself is evaluated and the history should narrate the maintenance process. I deliberately kept build output and binaries out of the repository (Git Lab p.6). The benefit appeared immediately in later labs: when [mistake] happened, I could [revert/diff/bisect] instead of reconstructing work, and the branch isolation meant my unfinished [feature] never blocked a releasable main."

Change request lab — writing the user story

Fill-in paragraph:

"In the change-request lab I initiated a change the way the course's change mini-cycle begins: with a change request expressed as a user story. I chose the JHotDraw feature [feature] and wrote: 'As a [user type], I want [goal] so that [reason]' (ChangeInitiation p.4–6). I kept the story small enough for a 3x5 card and checked it against the splitting rule — when my first draft covered both [aspect A] and [aspect B], I split it, because one story should carry one testable intention. Writing the story before touching code forced me to state the change in domain language rather than solution language, which later steered my concept location: the story's nouns ([noun], [noun]) became the concepts I searched for in the code. The acceptance criterion implied by 'so that [reason]' eventually became the given/when/then scenario in my BDD lab, so the same sentence travelled from initiation to verification."

Concept location lab — finding where the change goes

Fill-in paragraph:

"In the concept-location lab I located the code implementing [concept] in JHotDraw — the activity of finding the snippet where the change must be made. I combined two methodologies. First, dependency search: starting from [entry point], I followed the class dependencies toward the responsibility, asking of each class 'does this implement [concept]?'. Second, dynamic search with the IDE debugger (CLLab): I set breakpoints, exercised the feature by [user action] in the running editor, and observed which classes actually executed, yielding my initial class set: [Class1], [Class2], [Class3]. [Optional: I cross-checked with Featureous' feature-location views, which map executed code to features.] The two methods complement each other — static reading covers paths I failed to trigger, while the trace pinpoints code that demonstrably participates. The deliverable, my initial class set, became the input to impact analysis. The main lesson was partial comprehension: I never read all of JHotDraw; I read exactly enough to know where the change goes, which is the only comprehension maintenance budgets allow."

Impact analysis lab — estimating the blast radius

Fill-in paragraph:

"In the impact-analysis lab I extended the initial class set from concept location into an estimated impact set — the classes the change will actually touch. Following the marking algorithm of Figure 7.9 [Raj13] (AnalysisLab; ImpactAnalysis p.11–12), I took [InitialClass] as CHANGED, marked its neighbours in the class-interaction graph as NEXT, and inspected each: classes genuinely affected I marked CHANGED and propagated further; unaffected ones I marked UNCHANGED/INSPECTED. I paid special attention to propagating ('mailman') classes, which pass impact through without changing themselves. [Optional: I used JRipples to support the marking, since it implements exactly this propagation bookkeeping.] I considered both static dependencies and runtime interactions, because a class can be impacted through coordination without a compile-time reference (ImpactAnalysis p.5, p.8). My result — [N] classes across [packages], documented in the lab's Table 1 — shaped the plan: it told me the change was [small/medium/large] before I

wrote a line, and afterwards the actual diff confirmed the estimate was [accurate/over/under], which is precision and recall in practice (Actualization deck, Ericsson study)."

CI lab — building the pipeline

Fill-in paragraph:

"In the CI lab I set up continuous integration for my JHotDraw fork following the five classwork steps (CILab p.1). I added [maven.yml] under .github/workflows/ to define the pipeline, configured it to build the project with Maven on every pull request, wired shared dependencies from GitHub Packages through a .maven-settings.xml in the project root, and made the workflow execute my test suite automatically. The pipeline realises the deck's principles concretely: building each pull request is 'commit frequently, build every commit' (ContinuousIntegration p.5); the test execution makes the build self-testing (p.9); the fresh runner that only gets code from the repository kills 'it builds on my box' (p.6). I verified the safety net by pushing a deliberately broken test and watching the run go red before fixing it. The benefit compounded through every later lab: each refactoring and each feature merged only after a green build, so main remained releasable at all times and regressions surfaced within minutes of the commit that caused them."

Refactoring lab — improving structure without changing behaviour

Fill-in paragraph:

"In the refactoring lab I improved the maintainability of [Class/package] in JHotDraw without changing observable behaviour. I first identified smells: [methodName] violated G1 at [N] lines (BetterCode p.10), [method2] had [M] branch points violating G2 (p.16), and [ClassA]/[ClassB] shared a duplicated block of [K] lines violating G3 (p.23). I then applied catalog refactorings in small steps: Extract Method on [fragment], [Replace Conditional with Polymorphism / Introduce Parameter Object / Move Method] on [target], re-running the test suite after each step so behaviour-preservation was verified, not assumed — prefactoring before my feature change and postfactoring after it, in the course's terms. Each step was its own commit, e.g. '[commit message]', so the history documents the transformation. The measurable outcome: [unit] went from [N] to [n] lines and complexity [M] to [m], and the next change to that area ([which]) touched [fewer] classes. The lasting lesson is that refactoring is not cleanup after the fact but the enabling investment that makes the next change cheap."

Actualization lab — implementing and incorporating the change

Fill-in paragraph:

"In the actualization lab I implemented the change planned in the earlier phases — actualization being the phase where new functionality is actually coded and incorporated into the old code. I wrote [new class/method] to realise [user story] and incorporated it via [direct call / polymorphic extension / observer registration], choosing the incorporation point identified during concept location. I watched for change propagation: my modification of [Class] altered [its interface/contract], so the ripple reached [Neighbour], which I updated and marked off, continuing until no neighbour required change — the same propagation discipline as impact analysis, now with real edits. Where the deck's example extends behaviour by adding a [Pig]-style subclass rather than editing the consumer (Actualization p.7), I mirrored it: my [NewVariant] plugs into [Hierarchy] so the dispatching code stayed closed (OCP). I checked the result against my estimated impact set: predicted [N] classes, actually touched [M] — the comparison is my personal precision/recall measurement, and the gap taught me [lesson]."

Testing lab — unit tests with JUnit and AssertJ

Fill-in paragraph:

"In the testing lab (TestLab1) I built an automated unit-test suite around [feature/classes] in JHotDraw. I wrote JUnit tests with AssertJ assertions for [Class], designing inputs by equivalence partitioning and boundary-value analysis: partitions [list], boundaries [list] (Software Testing deck p.10–13). For the collaborator [dependency] I used a Mockito mock so the tests run without [database/GUI/filesystem], stay fast, and fail only when [Class] itself misbehaves. I followed Arrange–Act–Assert with one concept per test and behaviour-revealing names like [borrow_whenUnavailable_throws]. Where code resisted testing — [Class] constructed its own [dependency] — I refactored for testability by injecting the dependency through the constructor, experiencing first-hand that writing tests improves design (BetterCode p.54). The suite ([N] tests, running in [seconds]) was wired into the CI pipeline, where it now guards every later commit as a regression net: when my refactoring of [other class] broke [behaviour], the suite caught it before the merge, which is precisely the safety net refactoring presupposes."

BDD lab — executable scenarios with JGiven

Fill-in paragraph:

"In the BDD lab (TestLab2) I verified [feature] at the behaviour level by turning its user story into an executable specification. From the story 'As a [user], I want [goal] so that [reason]' I derived the scenario: Given [precondition], When [action], Then [outcome] — mapping story to scenario exactly as the lab's Figure 1 demonstrates (BDDLab p.1). I implemented it in JGiven with three stage classes — Given[State], When[Action], Then[Outcome] — passing state between stages via @ProvidedScenarioState and @ExpectedScenarioState, with AssertJ doing the assertion work in the Then stage (BDD p.10–16). I chose JGiven over a plain-text framework like Cucumber because the scenario is pure Java: it refactors with the production code and cannot drift from its step definitions, while still generating an HTML report that a non-programmer can read as living documentation (BDD p.6–9, p.17–18). The scenario closed the loop my change request opened: the same sentence that initiated the change now executes in the pipeline and proves the behaviour every build."